Examen Final: Se recomienda usar el browser Google Chrome; si no está en ese browser, no continúe; está a tiempo de cerrar la sesión e iniciarla nuevamente en Chrome.

Fecha de entrega 2 de sep en 10:00 **Puntos** 80 **Preguntas** 8
Disponible 2 de sep en 8:00 - 2 de sep en 10:00 casi 2 horas
Límite de tiempo 120 minutos

Instrucciones

Yo declaro que he sido informado y conozco las normas disciplinarias que rigen a la ESPOL, en particular el Código de Ética y el Reglamento de Disciplina. Al aceptar este compromiso de honor, reconozco y estoy consciente de que la presente evaluación está diseñada para ser resuelta de forma individual; que puedo comunicarme únicamente con la persona responsable de la recepción de la evaluación; y que al realizar esta evaluación no navegaré en otras páginas que no sean las páginas de Aula Virtual/plataforma de la evaluación; que no recibiré ayuda ni presencial ni virtual; que no haré consultas en libros, notas, ni apuntes adicionales u otras fuentes indebidas o no autorizadas por el evaluador; ni usaré otros dispositivos electrónicos o de comunicación no autorizados. Además, me comprometo a mantener encendida la cámara durante todo el tiempo de ejecución de la evaluación, y en caso de que el profesor lo requiera, tomar una foto de las páginas en las que he escrito el desarrollo de los temas y subirla a Aula Virtual/plataforma de la evaluación, como evidencia del trabajo realizado, estando consciente que el no subirla, anulará mi evaluación. Acepto el presente compromiso, como constancia de haber leído y aceptado la declaración anterior y me comprometo a seguir fielmente las instrucciones que se indican para la realización de la presente evaluación (incluyendo los requisitos de uso de la tecnología). Estoy consciente que el incumplimiento del presente compromiso, anulará automáticamente mi evaluación y podría ser objeto del inicio de un proceso disciplinario.

Si no está de acuerdo con el Compromiso de Honor, no continúe con el desarrollo de la evaluación; si procede a contestar las preguntas de la prueba, está aceptando el Compromiso de Honor.

Responda las siguientes 8 preguntas: Tiene 120 minutos para resolverlas.

Notar que sólo tiene un intento para responder cada pregunta; por lo tanto, revise bien antes de enviar la respuesta.

Suerte !

Historial de intentos

	Intento	Hora	Puntaje
MÁS RECIENTE	Intento 1	66 minutos	77.5 de 80

Puntaje para este examen: **77.5** de 80
Entregado el 2 de sep en 9:07
Este intento tuvo una duración de 66 minutos.

Pregunta 1

2.5 / 5 pts

El algoritmo de búsqueda "Primero el Mejor" utiliza una lista “Closed” para detectar lazos. Cuál sería el efecto de eliminar Closed y usar g(n) en su lugar (el componente g de la función heurística f) ? Seleccione todas las que crea verdaderas

espuesta correcta

- [ ] Podríamos perder la información para determinar si el nodo fue alcanzado a través de una ruta más corta

espuesta correcta

- [ ] Podríamos perder el camino hacia la solución final

¡Correcto!

- [x] Podríamos perder información, así como el historial de la búsqueda

¡Correcto!

- [x] Evitaría que el algoritmo regrese a un mejor nodo y regrese a la ruta de la solución final

Pregunta 2

5 / 5 pts

El algoritmo de búsqueda "Primero el Mejor" no verifica un nodo para probar si es el objetivo, hasta que se elimina de la lista Open. Sin embargo, esta prueba podría realizarse cuando se generen los nuevos estados y antes de que se agreguen a la lista Open. ¿Qué efectos podría tener este cambio en el algoritmo, en términos del tiempo y uso de memoria?

- ( ) El algoritmo decrementa el tiempo de búsqueda y el uso de la memoria
- ( ) El algoritmo incrementa el tiempo de búsqueda, pero decrementa el uso de memoria

¡Correcto!

- (●) El algoritmo incrementa el tiempo de búsqueda y el uso de la memoria
- ( ) El algoritmo decrementa el tiempo de búsqueda, pero incrementa el uso de memoria

Pregunta 3

5 / 5 pts

Cuál es el propósito de la instanciación universal y existencial ? Seleccione todas las que crea verdaderas

- [ ] Comprobar que una expresión cuantificada existencialmente, si se reemplaza por cualquier instancia, la expresión continúa siendo verdadera

¡Correcto!

- [x] El proceso de instanciación tiene el propósito de reducir las expresiones lógicas
- [ ] Comprobar que dos expresiones son sintácticamente equivalentes
- [ ] Comprobar que una expresión cuantificada universalmente, si se reemplaza por una instancia particular del rango, la expresión continúa siendo verdadera. La instancia particular es una función skolem

Pregunta 4

5 / 5 pts

Por qué son importantes los siguientes conceptos en Inteligencia Artificial? Seleccione todas las que crea verdaderas

- [ ] Recocido simulado porque son capaces de salir de mínimo local hacia un óptimo global en un período fijo de tiempo

¡Correcto!

- [x] A* porque garantiza encontrar la ruta más corta, tomando en cuenta el valor heurístico al objetivo y el costo real del recorrido, y permite realizar movimientos peores para alcanzar la solución

¡Correcto!

- [x] Resolución, porque nos permite inferir relaciones (reglas) de inferencia adicionales a las leyes de la lógica

- [ ] Cláusulas, porque luego de la transformación las expresiones son computacionalmente más eficaces

¡Correcto!

- [x] Unificación, porque la inferencia artificial se basa en la manipulación de símbolos y sus estructuras, y para poder reducirlos en la búsqueda de la solución, deben ser sintácticamente equivalentes

Pregunta 5

10 / 10 pts

El algoritmo de unificación no falla para cuál de los siguientes pares de expresiones?

- ( ) Pariente (Padre (x), y); Pariente (x, Padre (y))
- ( ) Pariente (Padre (x), Madre (y)); Pariente (x, Madre (y))
- ( ) Pariente (Padre (x), Padre (y)); Pariente (x, y)

¡Correcto!

- (•) Pariente (Padre (x), y); Pariente (v, z)

Pregunta 6

10 / 10 pts

Cuál de las siguientes afirmaciones es verdadera? Seleccione todas las que corresponda

¡Correcto!

- [x] En las redes RNN la propagación del error hacia atrás en el tiempo, utilizando la regla de la cadena, nos permite determinar la cantidad de error, con el que cada neurona contribuyó al error total de la red

- [ ] En las redes RNN hay una larga dependencia con las series en el pasado, para resolver su aprendizaje utilizamos el gradiente descendiente en el tiempo, lo que permite encontrar el error que cada neurona provocó en el pasado

- [ ] El problema de desvanecimiento del gradiente en las redes RNN, se debe a que la función de activación de las neuronas generalmente es un sigmoidal (como softmax), y la derivada tiende a cero

- [ ] La arquitectura que resuelve el problema del desvanecimiento de la derivada del sigmoidal es la arquitectura LSTM (Long Short Term Memory)

- [ ] A medida que las LSTM son más profundas, los gradientes que fluyen de regreso en el tiempo se vuelven más pequeños

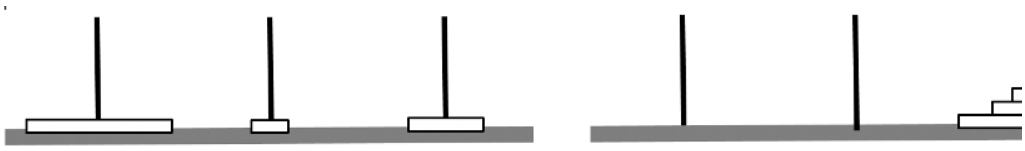
¡Correcto!

- [x] Una medida del número de capas escondidas necesarias en la RNN, es la longitud del contexto

Pregunta 7

15 / 15 pts

Considere la siguiente configuración de la Torre de Hanoi. La configuración de la izquierda es el estado inicial, y la configuración de la derecha es el estado objetivo

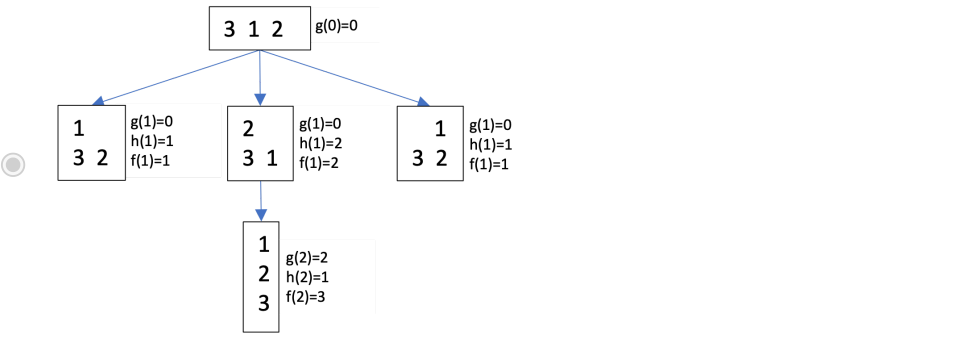


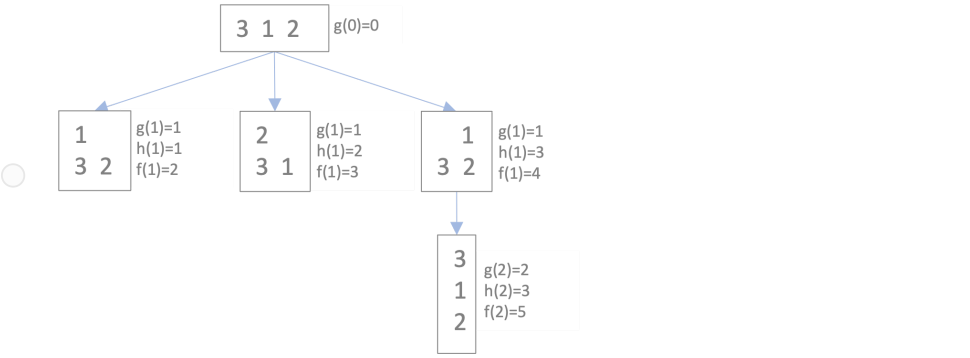
Estado actual: 3 1 2 Estado objetivo: 1 2 3

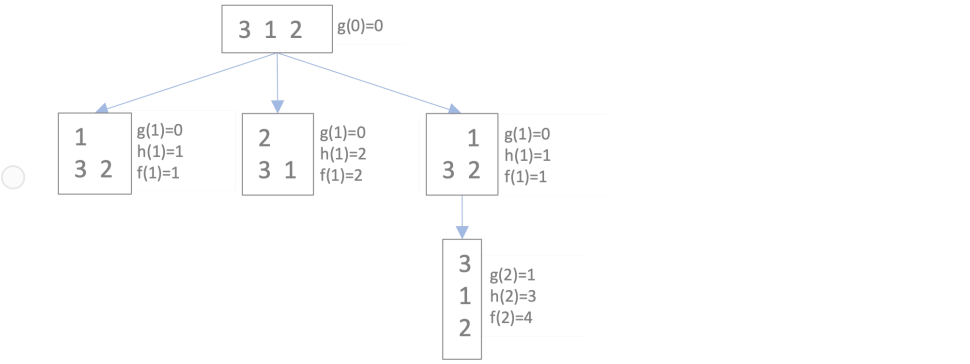
Si el costo de mover el disco más pequeño es 1, mover el disco mediano es 2 y mover el disco grande es 3; muestre cómo la estrategia de búsqueda Best-First, expande los primeros **dos** nodos del árbol de búsqueda, evitando estados repetidos.

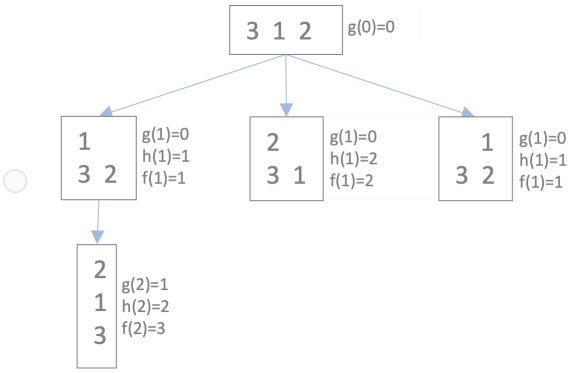
Para mostrar la estrategia de búsqueda tiene que dibujar el árbol (o crear en texto la estructura del árbol), en el que debe **mostrar claramente**: **el orden** de expansión de cada nodo; **el estado** de las torres **en cada nodo**, así como el **valor heurístico *f*** de cada nodo; se busca maximizar la función f(n), donde g(n) es el valor f(n-1); esto es, el valor f del estado anterior, y h(n) el valor de mover el disco; recuerde que en las torres de hanoi, no puede colocar un disco de tamaño mayor sobre uno de tamaño menor, y sólo puede mover un disco a la vez.

¡Correcto!









Pregunta 8

25 / 25 pts

Convertir la siguiente expresión en cláusulas:

$$\forall X\,(f(X) \Rightarrow \exists Y\,(m(X,Y))) \wedge \forall X\ \forall Z\,((a(X,Z) \vee b(X,Z)) \wedge c(Z,3))$$

- Ninguna de las anteriores
- i) $(f(X) \vee \neg\ (m(X,q(X))))$; ii) $((a(W,Z) \wedge b(W,Z))$; iii) $c(T,3))$

¡Correcto!

- (●) i) $(\neg\ f(X) \vee\ (m(X,q(X))))$; ii) $((a(W,Z) \vee b(W,Z))$; iii) $c(T,3))$
- i) $(\neg\ f(X) \vee\ (m(q(X))))$; ii) $((a(W,g(Z)) \vee b(T,g(Z)))$

Puntaje del examen: **77.5** de 80

Google Traducido al: español Mostrar texto original Traducir esta página al Español Opciones

II PAO 2021
Página de Inicio
Programa del curso
Módulos
Tareas
Calificaciones
Evaluaciones
Anuncios
Foros de discusión
Personas
Páginas
Zoom
Perusall
Registro de Asistencia

Lección # 3. Se recomienda usar el browser Google Chrome; si no está en ese browser, no continúe; está a tiempo de cerrar la sesión e iniciarla nuevamente en Chrome.

Fecha de entrega 14 de dic en 11:30 **Puntos** 10 **Preguntas** 3
Disponible 14 de dic en 11:15 - 14 de dic en 11:30 15 minutos **Límite de tiempo** 15 minutos

Detalles de la entrega:

Hora:	13 minutos
Puntaje actual:	10 de 10
se mantuvo el puntaje:	10 de 10

Instrucciones

Yo declaro que he sido informado y conozco las normas disciplinarias que rigen a la ESPOL, en particular el Código de Ética y el Reglamento de Disciplina. Al aceptar este compromiso de honor, reconozco y estoy consciente de que la presente evaluación está diseñada para ser resuelta de forma individual; que puedo comunicarme únicamente con la persona responsable de la recepción de la evaluación; y que al realizar esta evaluación no navegaré en otras páginas que no sean las páginas de Aula Virtual/plataforma de la evaluación; que no recibiré ayuda ni presencial ni virtual; que no haré consultas en libros, notas, ni apuntes adicionales u otras fuentes indebidas o no autorizadas por el evaluador; ni usaré otros dispositivos electrónicos o de comunicación no autorizados. Además, me comprometo a mantener encendida la cámara durante todo el tiempo de ejecución de la evaluación, y en caso de que el profesor lo requiera, tomar una foto de las páginas en las que he escrito el desarrollo de los temas y subirla a Aula Virtual/plataforma de la evaluación, como evidencia del trabajo realizado, estando consciente que el no subirla, anulará mi evaluación. Acepto el presente compromiso, como constancia de haber leído y aceptado la declaración anterior y me comprometo a seguir fielmente las instrucciones que se indican para la realización de la presente evaluación (incluyendo los requisitos de uso de la tecnología). Estoy consciente que el incumplimiento del presente compromiso, anulará automáticamente mi evaluación y podría ser objeto del inicio de un proceso disciplinario.

Si no está de acuerdo con el Compromiso de Honor, no continúe con el desarrollo de la evaluación; si procede a contestar las preguntas de la prueba, está aceptando el Compromiso de Honor.

Responda las siguientes 3 preguntas: Tiene 10 minutos para resolverlas.

Notar que sólo tiene un intento para responder cada pregunta; por lo tanto, revise bien antes de enviar la respuesta.

Suerte !

Este examen fue bloqueado en 14 de dic en 11:30.

Historial de intentos

	Intento	Hora	Puntaje
MÁS RECIENTE	Intento 1	13 minutos	10 de 10

Puntaje para este examen: **10** de 10
Entregado el 14 de dic en 11:30
Este intento tuvo una duración de 13 minutos.

Pregunta 1 — 3 / 3 pts

Cuáles de las siguientes afirmaciones son verdaderas? Seleccione todas las que considere verdaderas.

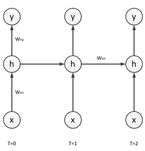
- [ ] El problema de desvanecimiento del gradiente en las redes RNN, se debe a que la función de activación de las neuronas generalmente es un sigmoidal (como softmax), y la derivada tiende a cero
- [ ] En las redes RNN hay una larga dependencia con las series en el pasado, para resolver su aprendizaje utilizamos el gradiente descendiente en el tiempo, lo que permite encontrar el error que cada neurona provocó en el pasado
- [x] En las redes RNN la propagación del error hacia atrás en el tiempo, utilizando la regla de la cadena, nos permite determinar la cantidad de error, con el que cada neurona contribuyó al error total de la red — ¡Correcto!
- [x] Una medida del número de capas escondidas necesarias en la RNN, es la longitud del contexto — ¡Correcto!

Pregunta 2 — 4 / 4 pts

La siguiente figura muestra una RNN con una entrada X, una capa oculta h, con la función de activación logística, y una capa de salida Y, con función la activación lineal. La RNN está desenrollada en el tiempo para t=0, 1 y 2.

Asuma que los parámetros iniciales, asignados aleatoriamente son: W_{xh} = 0.8, W_{hh} = 1.5, W_{hy} = −0,5, $_{sesgo}$ h = 1,2, $_{sesgo}$ Y = 0,8.

Si la entrada X tiene los valores 6, 2, −1, en los pasos de tiempo t= 0,1,2 respectivamente. Cuál es la salida Y en el paso de tiempo t=1 ?.



- [ ] 4.3
- [ ] - 0.21
- [x] 0.31 — ¡Correcto!
- [ ] 0.99

Pregunta 3 — 3 / 3 pts

Cuál de las siguientes afirmaciones es verdadera? Seleccione todas las que crea verdaderas

- [ ] En las redes RNN hay una larga dependencia con los time-steps en el pasado, y una técnica para actualizar los parámetros es calcular la probabilidad en la salida y trasladarla hacia atrás con back-propagation through time
- [x] Una medida del número de capas escondidas necesarias en las RNN, es la longitud del contexto a recorder — ¡Correcto!
- [ ] La arquitectura que resuelve el problema de desvanecimiento de la derivada del sigmoidal es la arquitectura LSTM (Long Short Term Memory)
- [x] En las RNN, al aplicar la regla de la cadena, si cualquiera de los gradientes se acerca a 0, entonces todos los siguientes gradientes se acercarán a 0 exponencialmente de manera rápida, debido a la multiplicación — ¡Correcto!

Puntaje del examen: **10** de 10

Texto original
Sugiere una traducción mejor

Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Semestre 2021 – II

Agenda

1. Redes neuronales recurrentes
   1.1 Ejemplo procesamiento de lenguaje con RNN
   1.2 Aprendizaje en las RNN - BPTT
   1.3 Desvanecimiento del gradiente

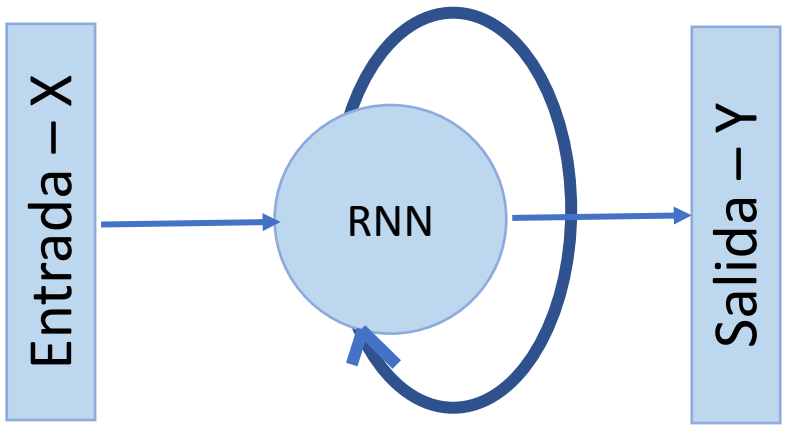
Redes neuronales recurrentes

Modelado del lenguaje

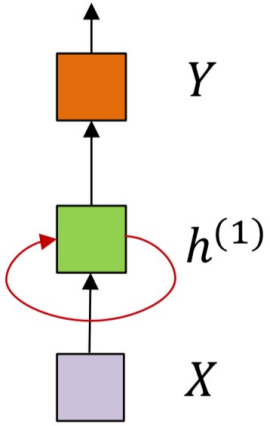
- Las RNN utilizan información secuencial y se denominan recurrentes porque realizan la misma tarea para cada elemento de la secuencia, y la salida depende de los cálculos anteriores
- Idealmente las RNN podrían utilizar la secuencia de información arbitrariamente larga del pasado, pero en la práctica, se limitan a unos pocos pasos en el pasado

Redes neuronales recurrentes

- Por ejemplo: Supongamos una RNN, con la que queremos predecir caracteres
- Supongamos que tenemos la palabra “feliz”; y, luego de ingresar las primeras 4 letras: “f, e, l, i”, queremos que la RNN prediga la siguiente letra: “z”
- En este ejemplo el vocabulario tiene 4 letras: {f, e, l, i}; este vocabulario se denomina “Corpus”, en la práctica, este puede llegar a tener millones de palabras (por ejemplo, la base completa de Wikipedia)
- Veamos cómo esta estructura es utilizada para predecir la quinta letra en la palabra “feliz”



Redes neuronales recurrentes

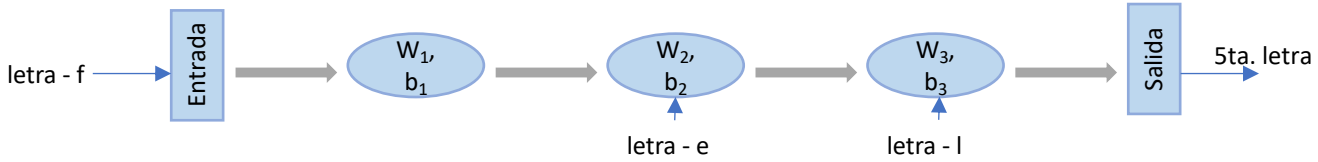


Procedimiento:

1. Ingresamos un caracter a la vez (un time step), es decir X_t se suministra a la red
2. Luego calculamos su estado actual utilizando la combinación de la entrada actual y el estado anterior, es decir, calculamos h_t
3. Este h_t actual se convierte en h_{t-1} para el siguiente paso de tiempo
4. Podemos seguir tantos pasos de tiempo como el problema lo requiera y combinar la información de todos los estados anteriores
5. Una vez que se completan todos los pasos de tiempo, se utiliza el estado actual final para calcular la salida Y_t
6. La salida se compara con la salida deseada y se calcula el error
7. Luego, el error se propaga hacia atrás de la red para actualizar los pesos... Al final la red está entrenada

Redes neuronales recurrentes

- Ingresamos las letras en secuencia:
  - En nuestro ejemplo la letra ”f", la cual no tiene nada que la preceda
  - Luego ingresamos la letra ”e”
  - En el momento que suministramos la letra ”e", se aplica la fórmula de recurrencia a esa letra, y al estado anterior de la capa escondida, en este caso la letra ”f”
  - Este proceso de ingresar cada letra se conoce como pasos de tiempo de la entrada (time steps)
  - Si en el momento t, la entrada es ”e", en el tiempo $t - 1$, la entrada fue ”f"
  - Para obtener el nuevo estado, la fórmula de recurrencia se aplica tanto a “e” como a “f”, así:



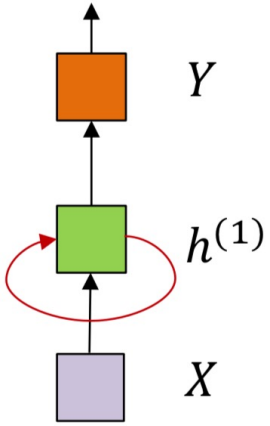
Redes neuronales recurrentes

- Veamos el paso “forward” en nuestro ejemplo:

f		e		l		i
1		0		0		0
0		1		0		0
0		0		1		0
0		0		0		1

- Las entradas están en formato: one-hot-encoded
- El vocabulario completo es: {f, e, l, i}

Redes neuronales recurrentes



- Supongamos una capa con 3 neuronas de ancho
- Entonces necesitamos una matriz W_{xh} de dimensión 3 (neuronas) x 4 (tamaño del corpus)

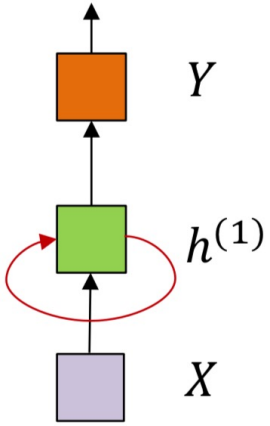
W_{xh}			
0.287027	0.84606	0.572392	0.486813
0.902874	0.871522	0.691079	0.18998
0.537524	0.09224	0.558159	0.491528

W_{hy}		
0.37168	0.974829459	0.830034886
0.39141	0.282585823	0.659835709
0.64985	0.09821557	0.334287084
0.91266	0.32581642	0.144630018

W_{hh}
0.427043

Bias
0.56700

Redes neuronales recurrentes



- Para la letra “f”, como no tenemos un estado previo, el estado es [0, 0, 0], entonces el estado actual, en el paso 0, es:

$$h_0 = tanh(W_{hh}h_{0-1} + W_{xh}X_0)$$

- En el cálculo $W_{hh}h_{t-1}$, no olvidar el bias: $W_{hh}h_{t-1} + bias$

W_{hh}
0.427043

$\times$

h_{0-1}
0
0
0

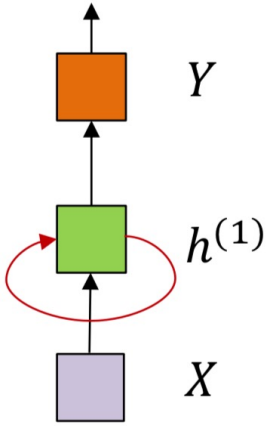
^{T} $+$

Bias
0.56700

$=$

$W_{hh}h_{0-1}$+bias
0.567001
0.567001
0.567001

Redes neuronales recurrentes

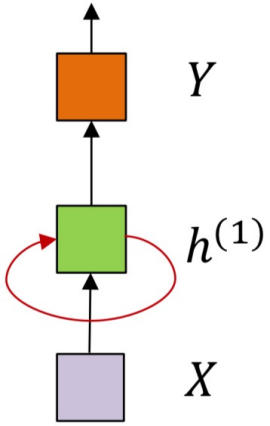


- Entonces: $h_0 = tanh(W_{hh}h_{0-1} + W_{xh}X_0)$, es:

$$h_0 = \tanh\left\{\begin{bmatrix}0.567001\\0.567001\\0.567001\end{bmatrix} + \begin{bmatrix}0.287027359\\0.902874425\\0.537523791\end{bmatrix}\right\}$$

$$h_0 = \begin{bmatrix}0.69316804\\0.89955366\\0.80211840\end{bmatrix}$$

Redes neuronales recurrentes

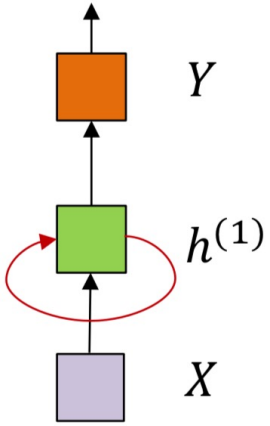


- Entonces el calculo de $h_1 = tanh(W_{hh}h_0 + W_{xh}X_1)$, es:

$$h_1 = \tanh\left\{\begin{bmatrix}0.863013\\0.951149\\0.90954\end{bmatrix} + \begin{bmatrix}0.846060\\0.871522\\0.092240\end{bmatrix}\right\}$$

$$h_1 = \begin{bmatrix}0.93653372\\0.94910403\\0.76234057\end{bmatrix}$$

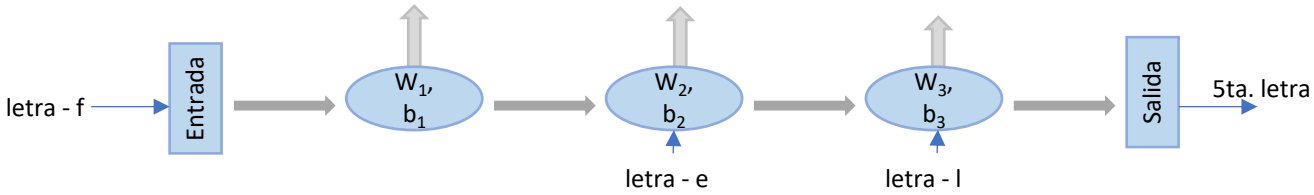
Redes neuronales recurrentes



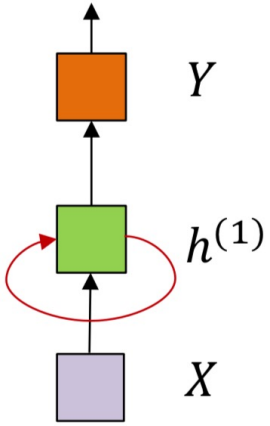
- Entonces el calculo de $h_1 = tanh(W_{hh}h_0 + W_{xh}X_1)$, es:

$$h_1 = \tanh\left\{\begin{bmatrix}0.863013\\0.951149\\0.90954\end{bmatrix} + \begin{bmatrix}0.846060\\0.871522\\0.092240\end{bmatrix}\right\}$$

$$h_1 = \begin{bmatrix}0.93653372\\0.94910403\\0.76234057\end{bmatrix}$$



Redes neuronales recurrentes



- Finalmente, calculamos la probabilidad de las letras en el vocabulario, aplicando la función softmax
- La letra en particular que el sistema recomendaría, en nuestro ejemplo, después de “e”, sería la letra que obtenga la mayor probabilidad, luego de aplicar softmax
- En nuestro ejemplo:

Probabilidades = Softmax

Y_1
1.90607732
1.13779113
0.95666016
1.27422602

=

Probabilidades
0.419748
0.194682
0.162429
0.223141

Redes neuronales recurrentes

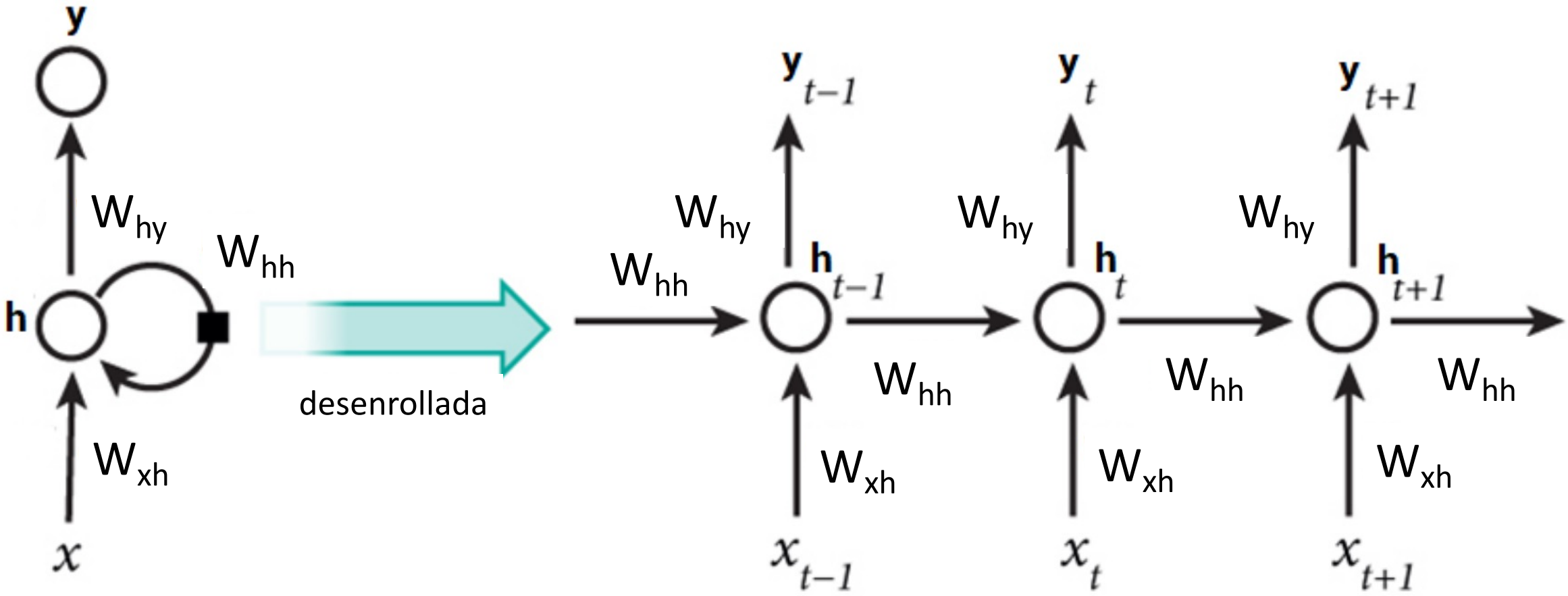
- Estas probabilidades lo que nos dicen es que la letra después de "e" debe ser “f”, que tiene la probabilidad más alta
- La predicción no es la mejor, puesto que apenas hemos empezado a entrenar la red
- Acabamos de mostrarle dos letras. Así que prácticamente no ha aprendido nada todavía
- Una vez que concluya la última letra “i”, cómo se actualizan los pesos?
- Como en los modelos anteriores estas redes, “aprenden” también a través de backpropagation, que en RNN se lo llama “Back propagation through time” (BPTT)

Redes neuronales recurrentes

- Back propagation en RNN – BPTT
  - Para entender y visualizar la propagación hacia atrás, desenrollemos la capa escondida de la red en todos los pasos de tiempo
  - Recuerde que en una RNN podemos tener o no salidas en cada paso de tiempo
  - En el caso de la propagación hacia delante (forward propagation), las entradas ingresan y avanzan en cada paso de tiempo
  - En la propagación hacia atrás (back propagation), de manera figurativa, retrocedemos en el tiempo para cambiar los pesos, por eso se conoce a esta técnica como propagación hacia atrás a través del tiempo (Back Propagation Through Time - BPTT).

Redes neuronales recurrentes

- RNN desenrollada



Redes neuronales recurrentes

- En resumen, el algoritmo BPTT:

1. Calculamos el error utilizando la salida actual y la salida esperada
2. Desenrollamos todos los pasos de tiempo
3. Para la red desenrollada, se calcula el gradiente para cada paso de tiempo con respecto a los pesos
4. Como los pesos son los mismos para todos los pasos de tiempo, los gradientes se repiten en cada paso
5. Los pesos se actualizan para las neuronas recurrentes y las capas de salida

Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

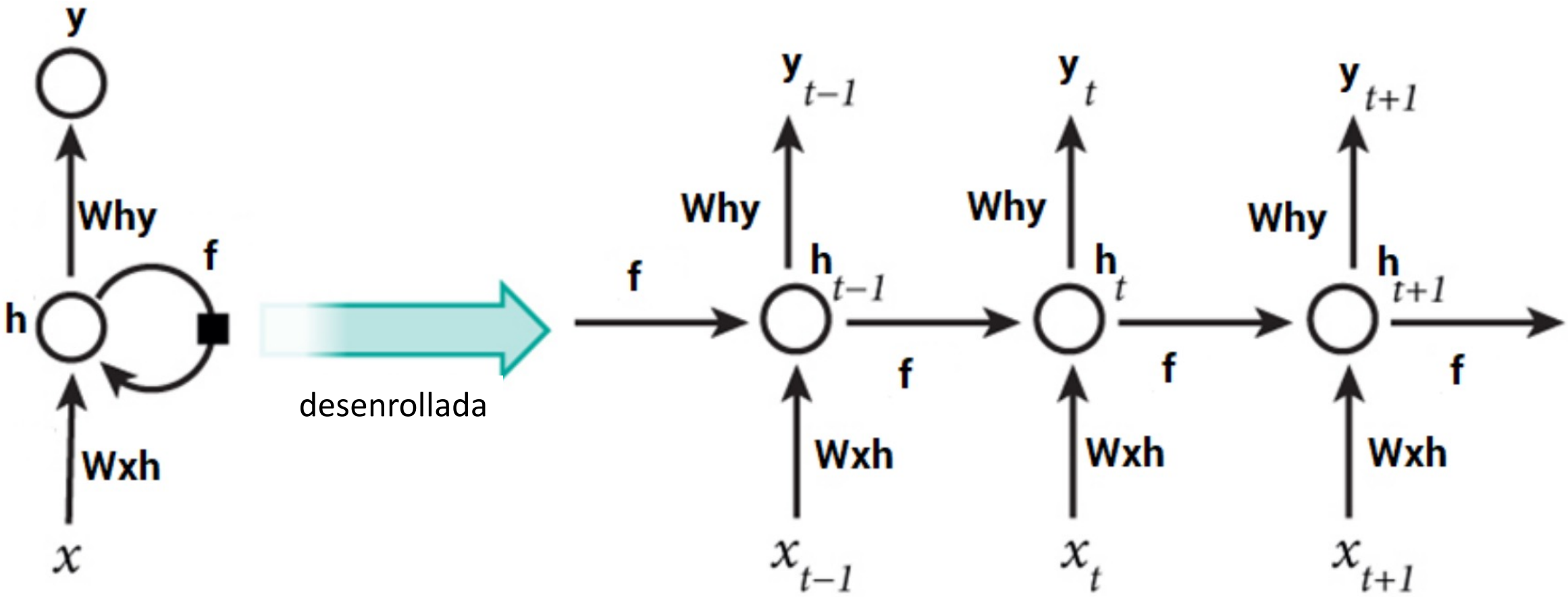
Semestre 2021 – II

Agenda

1. Redes neuronales recurrentes
   1.1 Aprendizaje en las RNN - BPTT
   1.2 Desvanecimiento del gradiente
   1.3 Introducción a las Redes recurrentes tipo Long Short Term Memory (LSTM)

Redes neuronales recurrentes

- RNN desenrollada



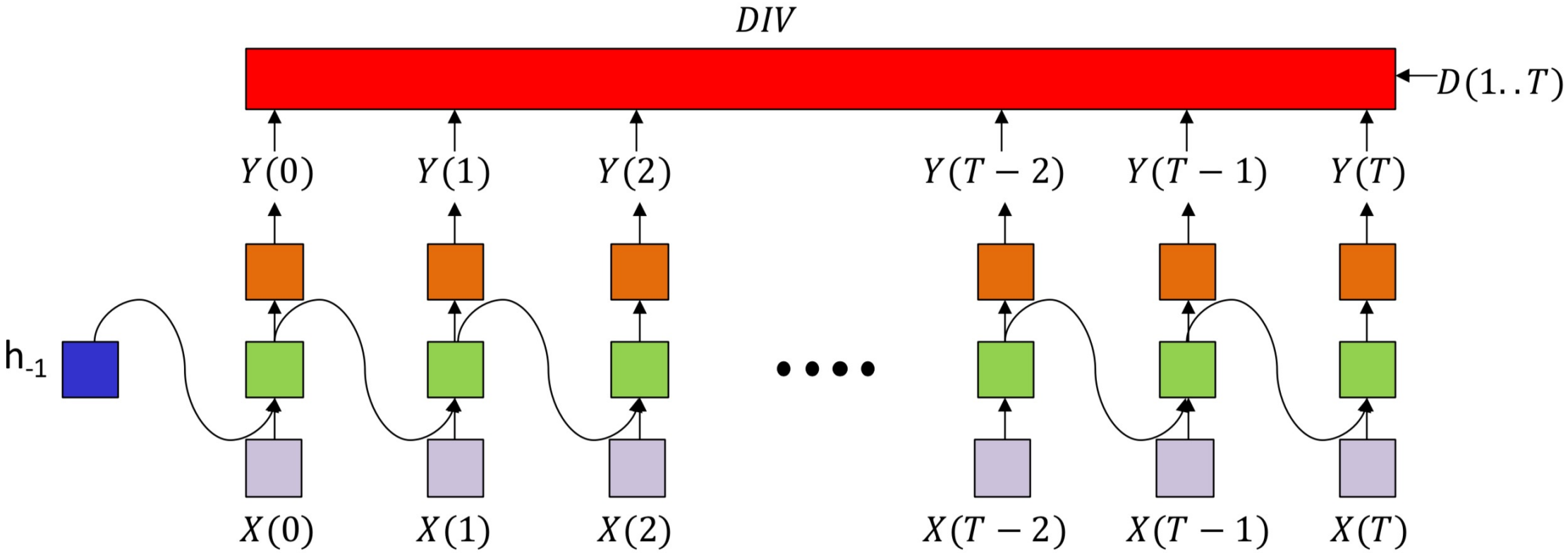
Redes neuronales recurrentes

- En resumen, el algoritmo BPTT:

1. Calculamos el error utilizando la salida actual y la salida real
2. Desenrollar todos los pasos de tiempo
3. Para la red desenrollada, se calcula el gradiente para cada paso de tiempo con respecto a los pesos
4. Como los pesos son los mismos para todos los pasos de tiempo, los gradientes se repiten en cada paso
5. Los pesos se actualizan para las neuronas recurrentes y las capas de salida

Redes neuronales recurrentes

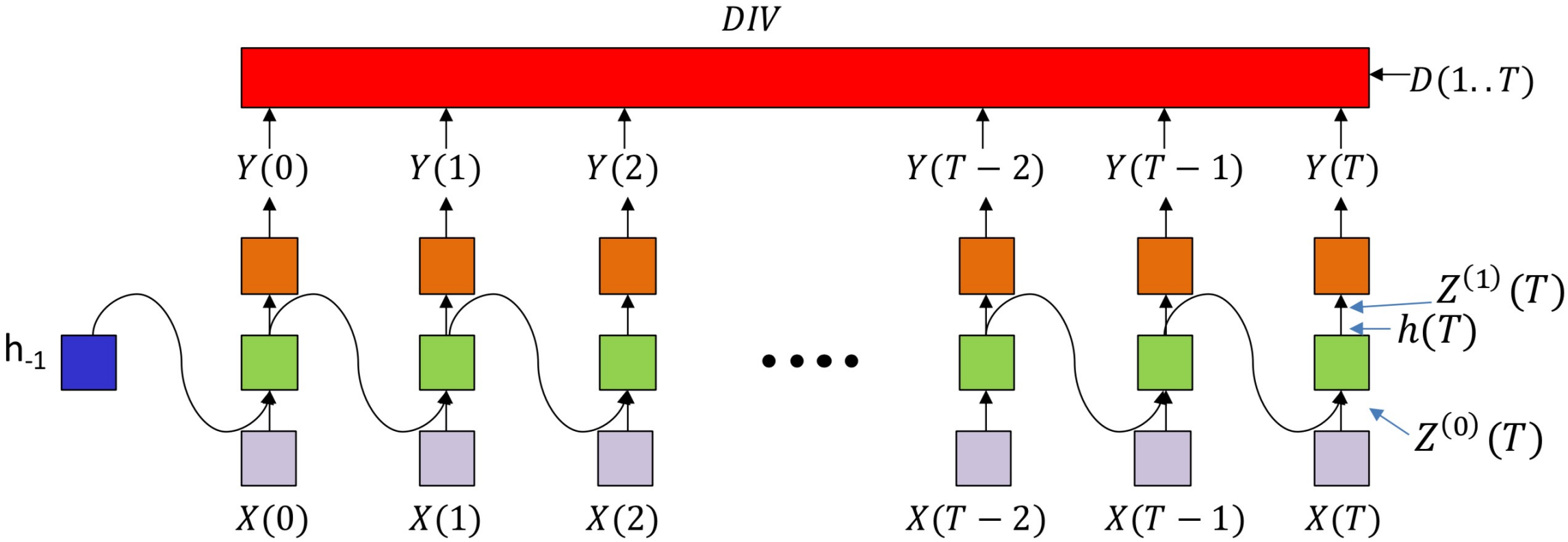
- Cómo se calcula el gradiente en BPTT



- La divergencia se calcula entre la secuencia de salidas reales de la red y la secuencia de salidas deseadas

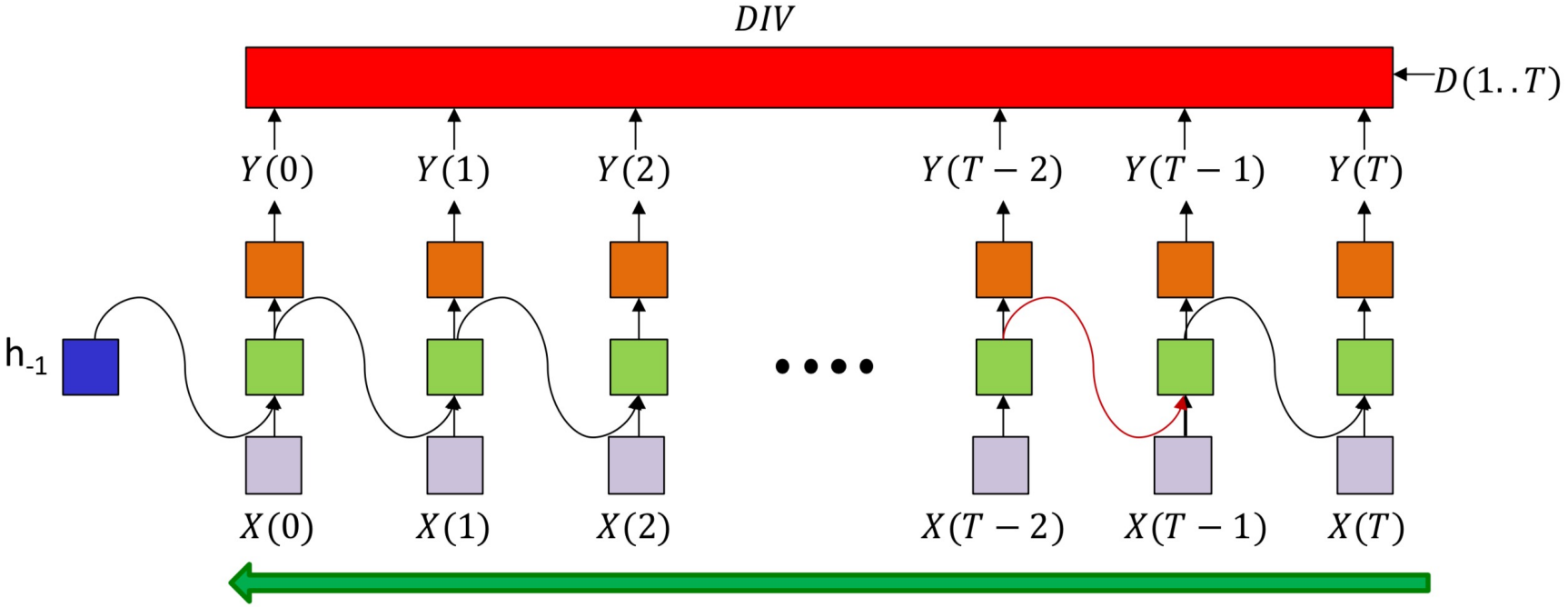
Redes neuronales recurrentes

- Cómo se calcula el gradiente en BPTT



Redes neuronales recurrentes

- Cómo se calcula el gradiente en BPTT



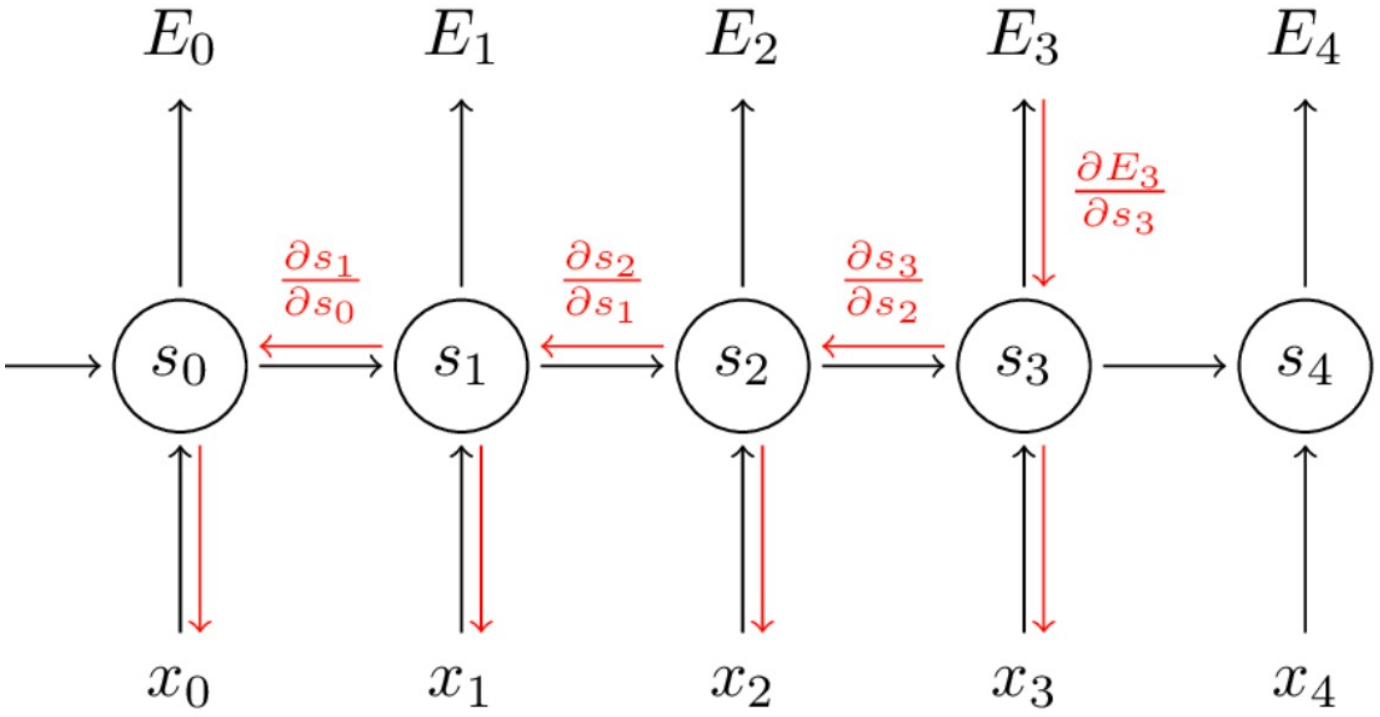
$$\frac{dDIV}{dh_{-1}} = \sum_j w_{ij}^{(11)} \frac{dDIV}{dZ_j^{(1)}(0)}$$

$$\frac{dDIV}{dw_{ij}^{(0)}} = \sum_t \frac{dDIV}{dZ_j^{(0)}(t)} X_i(t)$$

$$\frac{dDIV}{dw_{ij}^{(11)}} = \sum_t \frac{dDIV}{dZ_j^{(0)}(t)} h_i(t-1)$$

Redes neuronales recurrentes

- En otras palabras, debido a que W se usa en cada paso hasta la salida, necesitamos propagar los gradientes de regreso desde la salida a través de la red hasta $t = 0$
- Por ejemplo: la propagación del error desde $t = 3$ es:



Redes neuronales recurrentes

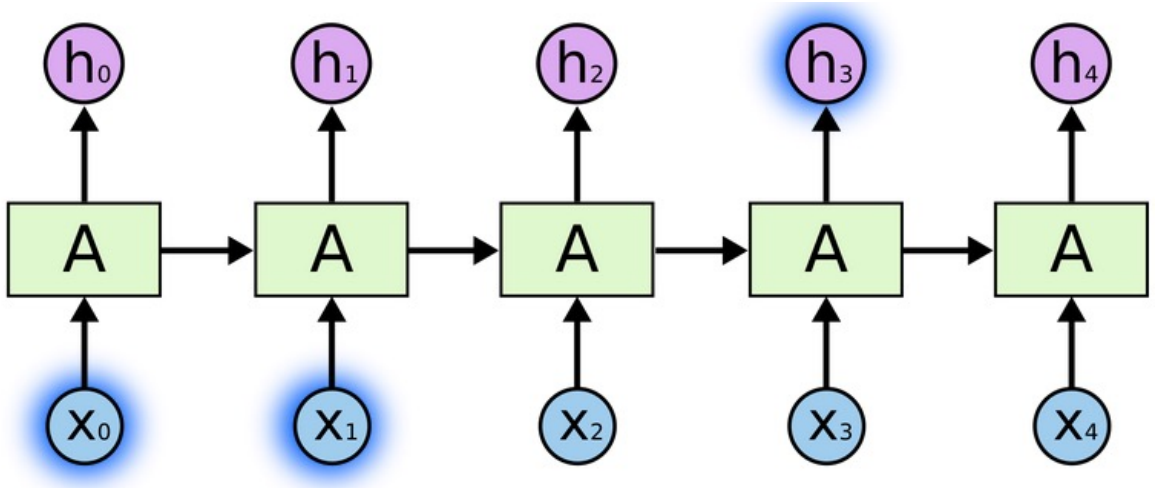
- Para calcular el error después del tercer paso de tiempo con respecto al primero:

$$\frac{\partial E}{\partial W} = \frac{\partial E}{\partial Y_3} * \frac{\partial Y_3}{\partial h_3} * \frac{\partial h_3}{\partial Y_2} * \frac{\partial Y_2}{\partial h_1} \ldots$$ hasta llegar a W

- Vemos que hay una larga dependencia
- Al aplicar la regla de la cadena, si cualquiera de los gradientes se acerca a 0, entonces todos los gradientes se acercarán a 0 exponencialmente de manera rápida, debido a la multiplicación
- Esos estados ya no ayudarían a la red a aprender nada
- Esto se conoce como el problema de desvanecimiento del gradiente

Redes neuronales recurrentes

- Las redes RNN son efectivas cuando la brecha entre la información relevante y el lugar en donde se la necesita es corta



- Esto provoca que las RNN a veces fallen en “entender” el contexto
- Estas redes no pueden recordar algo que se dijo mucho antes, para poder predecir el presente

Long Short Term Memory Networks - LSTM

- Las redes Long Short Term Memory - LSTM, resuelven este problema, son un tipo especial de RNN
- Son capaces de aprender dependencias a largo plazo
- Fueron propuestas por Hochreiter y Schmidhuber en 1997
- Funcionan bien en una gran variedad de problemas, y hoy representan el estado del arte en RNN
- Las LSTM también tienen esta estructura tipo secuencial, pero el módulo recurrente tiene una estructura diferente; en lugar de tener una sola capa de red neuronal, tienen múltiples capas, que interactúan de una manera particular
- Las LSTM tienen 3 puertas: Una de entrada, una de olvido y una de salida

Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

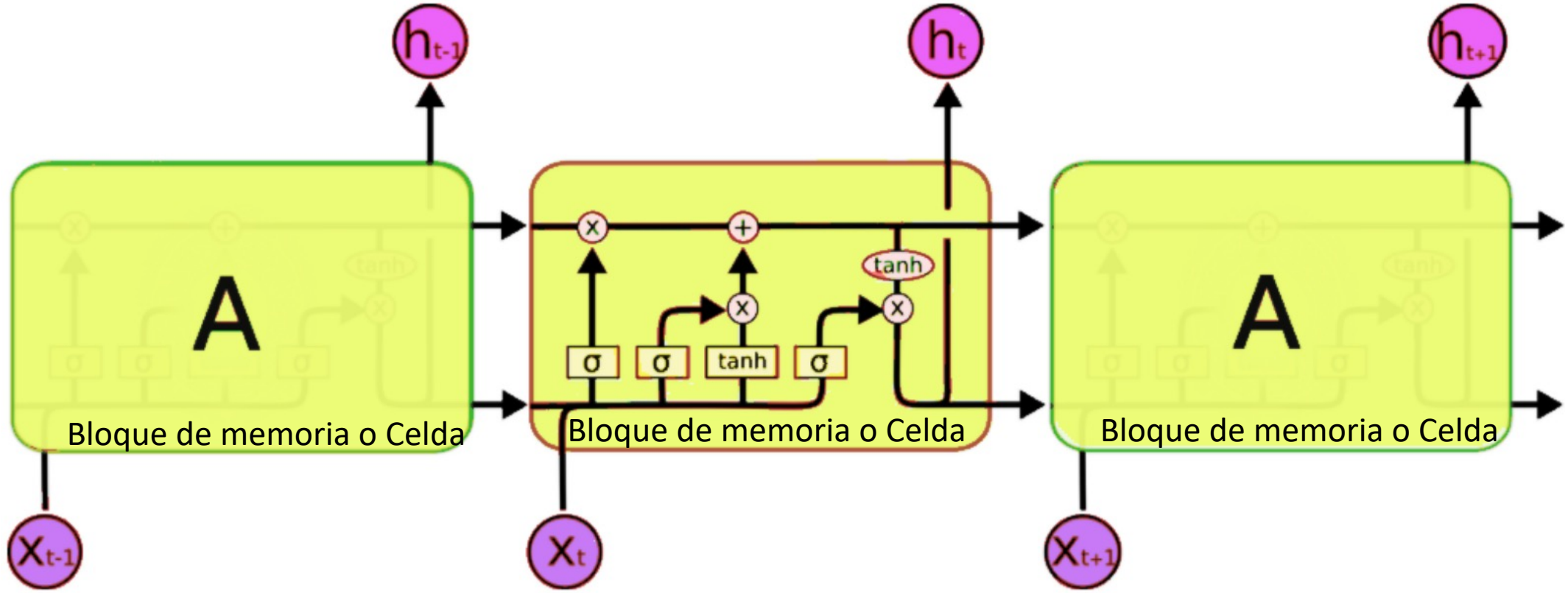
Facultad de Ingeniería en Electricidad y Computación – FIEC

Semestre 2021 – II

Agenda

- Redes LSTM
  - Celdas, gates
  - Recurrencia
  - Estado de las celdas

Long Short Term Memory Networks- LSTM

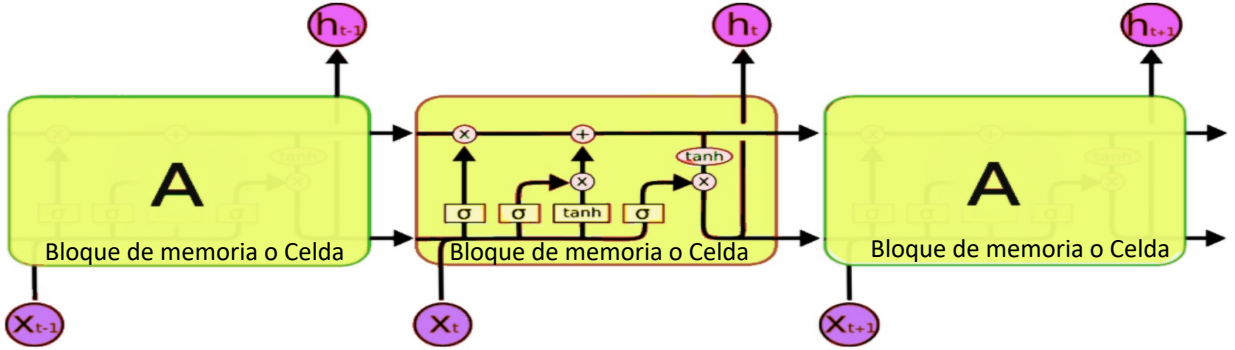


- Las LSTM tienen un flujo de control similar al de una RNN
- Procesa los datos que ingresan a medida que se propaga en la red, la diferencia son las operaciones dentro de las celdas de LSTM

Fuente: Andrej Karpathy - http://karpathy.github.io/2015/05/21/rnn-effectiveness/

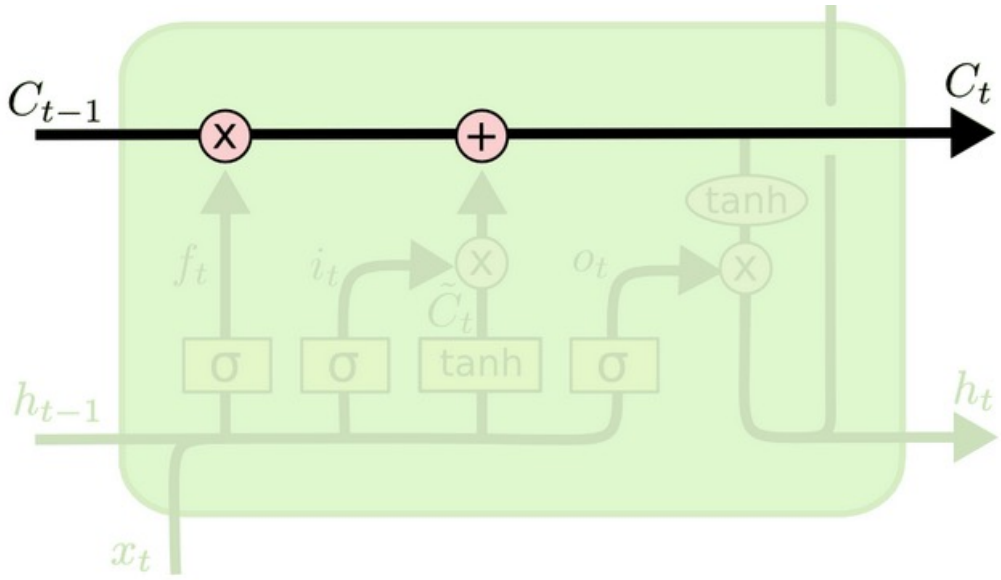
Long Short Term Memory Networks - LSTM

- La Idea principal
  - El concepto central de LSTM es el estado de la celda, y sus puertas
  - El estado de la celda es la "memoria" de la red
  - La celda puede transportar información relevante a lo largo del procesamiento de la secuencia
  - Por lo tanto, la información de los pasos de tiempo anteriores pueden trasladarse a los pasos de tiempo posteriores, reduciendo los efectos de la memoria a corto plazo



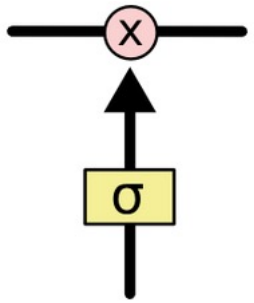
Long Short Term Memory Networks- LSTM

- Entonces un aspecto importante en las redes LSTM es el estado de la celda, la línea horizontal que recorre la parte superior del diagrama
- El estado de la celda recorre toda la cadena de celdas



Long Short Term Memory Networks - LSTM

- La LSTM tiene la capacidad de eliminar o agregar información al estado de la celda, regulada por estructuras llamadas compuertas (puertas o gates)
- Las puertas son una forma de pasar información. Estas son RN de una capa con una función de activación tipo sigmoidal y una operación de multiplicación pointwise
- Esta capa sigmoidal produce números entre 0 y 1, y decide sobre la cantidad de información que cada componente debe dejarse pasar
- 0 significa "no dejar pasar nada", 1 significa "dejar pasar todo”
- Una LSTM tiene 3 de estas puertas, para proteger y controlar el estado de la celda



Long Short Term Memory Networks - LSTM

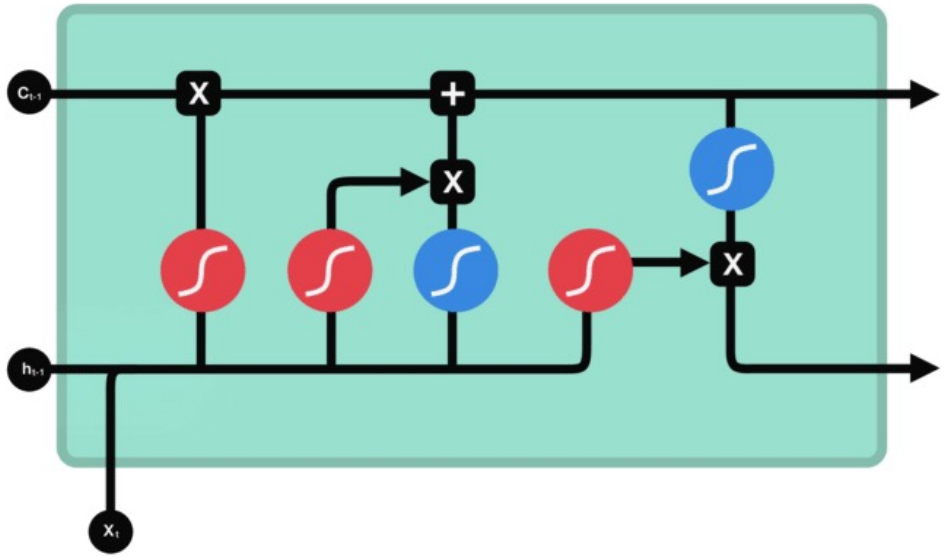
- El primer paso en la red LSTM es decidir qué información va a eliminar (si hay algo que eliminar) del estado de la celda
- Esta decisión la toma la capa sigmoidal llamada ”puerta de olvido”
- Esta puerta mira a h_{t-1} y x_t, y genera un número entre 0 y 1 para cada valor en el estado de celda C_{t-1}
- Recordar, 1 representa "mantener toda la información”; y, 0 "deshacerse de toda la información”

Long Short Term Memory Networks- LSTM

- Puerta de olvido
  - Esta puerta decide qué información se debe desechar o guardar
  - La información del estado anterior y la información de la entrada actual se pasan a través de la función sigmoidal
  - Los valores salen entre 0 y 1. Cuanto más cerca de 0 significa olvidar, y más cerca de 1 significa mantener

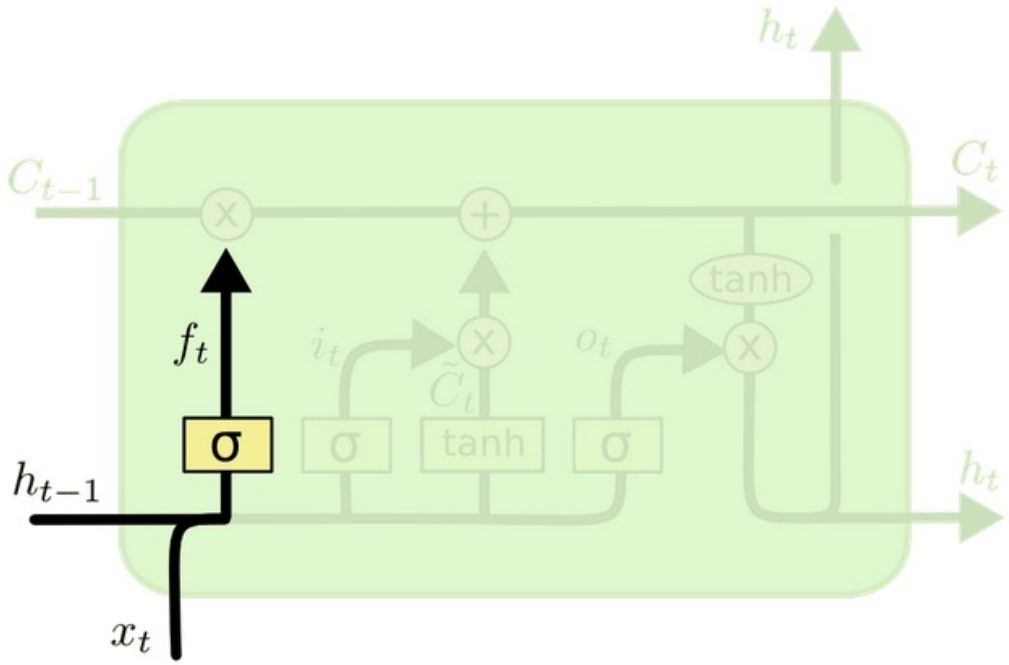
C_{t-1} estado previo de la celda

f_t salida de la puerta de olvido



Long Short Term Memory Networks - LSTM

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$



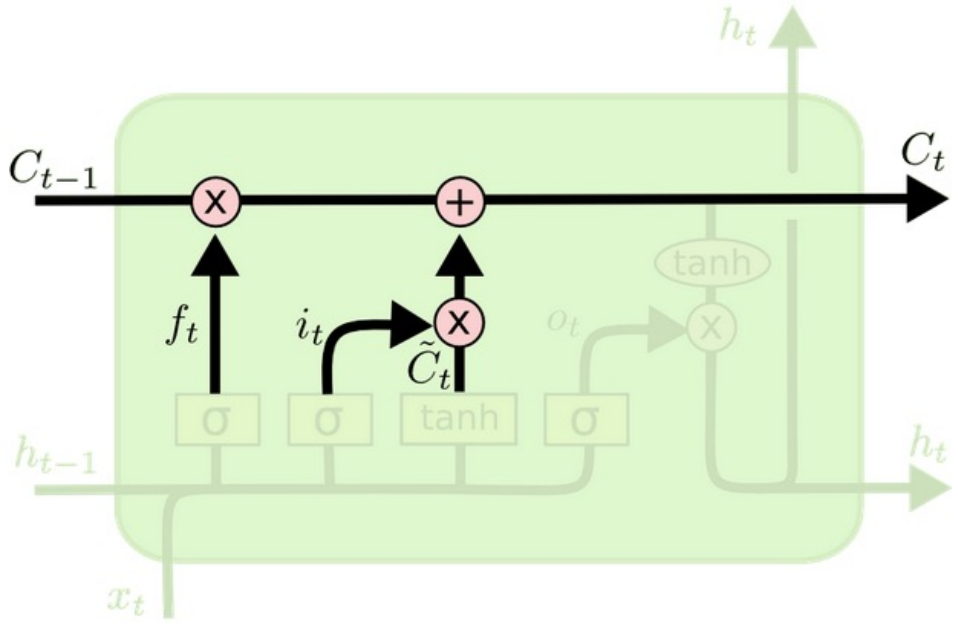
Long Short Term Memory Networks - LSTM

- Puerta de entrada
  - La puerta de entrada actualiza el estado de la celda
  - Primero, el estado anterior y la entrada actual pasan a través de una función sigmoidal
  - La sigmoidal decide qué valores se actualizan, transformándolos entre 0 y 1 (0 significa que no es importante y 1 significa importante)
  - También, el estado anterior y la entrada actual pasan por la función tanh para ajustar los valores entre -1 y 1, y ayudar a regular la red
  - Luego, se multiplica la salida de tanh con la salida sigmoidal
  - La salida sigmoidal determina qué información es importante mantener de la salida de tanh

Long Short Term Memory Networks- LSTM

- En este estado también se agrega información, tal como se decidió en los pasos anteriores

$$C_t = f_t * C_{t-1} + i_t * \widetilde{C}_t$$

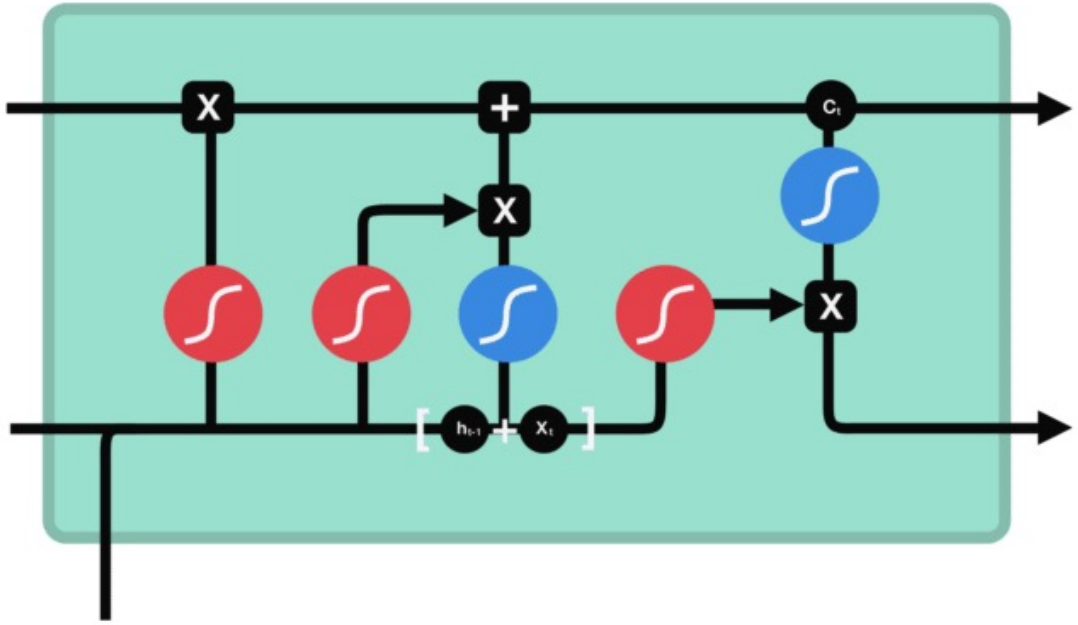


Long Short Term Memory Networks- LSTM

- Puerta de salida
  - La puerta de salida decide cuál debe ser el siguiente estado de la celda
  - Recuerde que el estado de la celda contiene la información de las entradas anteriores. El estado de la celda también se usa para predicciones
  - Primero, pasamos el estado anterior y la entrada actual a una función sigmoidal
  - Luego pasamos el estado de celda recién modificado a la función tanh
  - Multiplicamos la salida de tanh con la salida sigmoide para decidir qué información debe llevar el estado oculto
  - La salida es el nuevo estado oculto
  - El nuevo estado de la celda se transfieren al siguiente paso de tiempo

Long Short Term Memory Networks- LSTM

- En resumen:
  - La puerta de olvido decide qué es relevante mantener de los pasos anteriores
  - La puerta de entrada decide qué información es relevante agregar del paso actual
  - La puerta de salida determina cuál debería ser el siguiente estado oculto de la red



C_{t-1} estado previo de la celda

f_t salida de la puerta de olvido

i_t salida de la puerta de entrada

$\tilde{C}_t$ candidatos

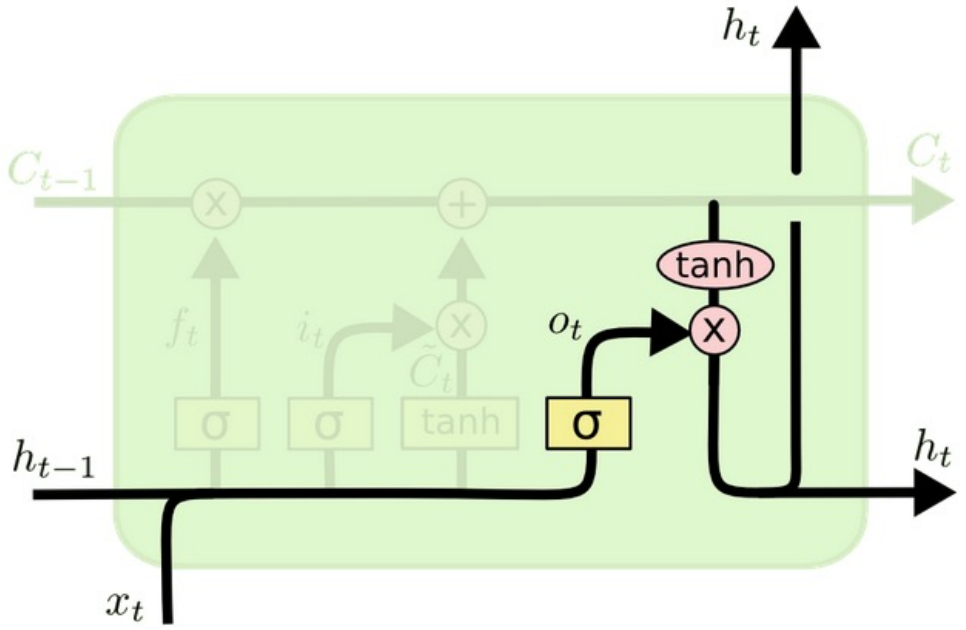
C_t nuevo estado de la celda

o_t salida de la puerta de salida

h_t estado oculto

Long Short Term Memory Networks - LSTM

- $o_t = \sigma\left(W_o \cdot [h_{t-1}, x_t] + b_o\right)$
- $h_t = o_t * \tanh(C_t)$



Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

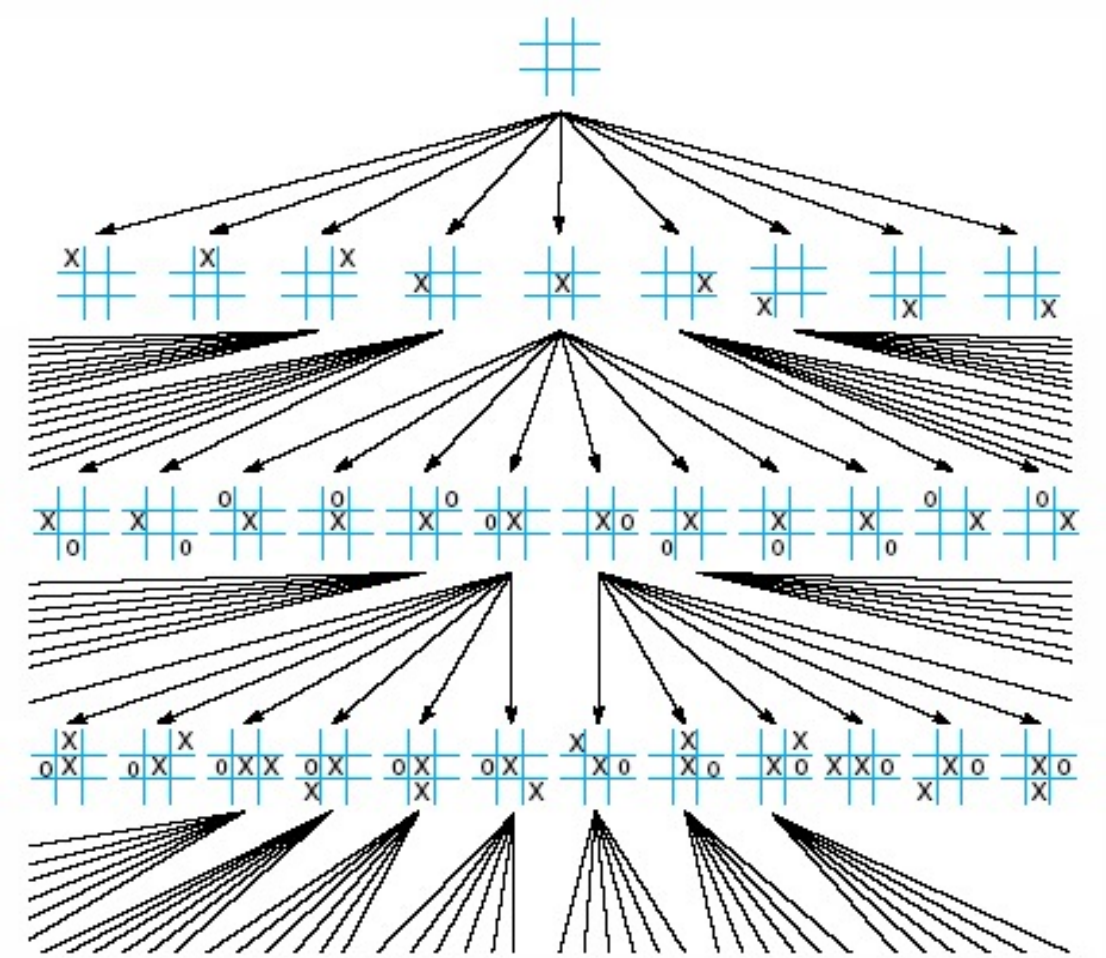
Facultad de Ingeniería en Electricidad y Computación – FIEC

Semestre 2021 – II

Agenda

- Técnicas para resolver problemas de IA
- Búsqueda a ciegas
  - Generar y probar (Generate and Test)
  - Breath Search
  - Depth Search

Ejemplo: TRES EN RAYA



Técnicas para resolver problemas en IA

- Este ejemplo expone 3 características importantes de las técnicas utilizadas en IA:
  - **Búsqueda**: proporciona un mecanismo para resolver un problema donde no existe un método directo.
  - **Uso de conocimiento**: proporciona un mecanismo para resolver problemas complejos utilizando las estructuras e información existentes de los objetos involucrados.
  - **Abstracción**: proporciona un mecanismo para concentrarse en aquellos aspectos que son importantes para la solución del problema, y separa aquellos que no son importantes y que podrían hacer que el proceso colapse.

Técnicas para resolver problemas en IA

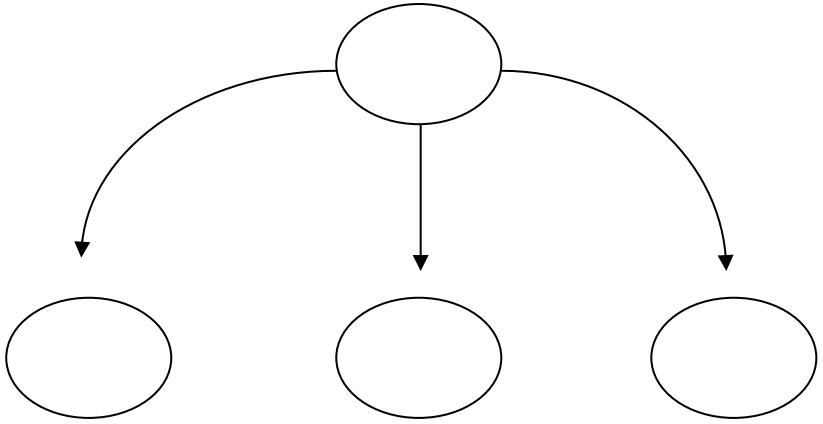
- Las primeras investigaciones en IA mostraron que el comportamiento inteligente necesita conocimiento.

- El conocimiento tiene algunas propiedades indeseables, como:

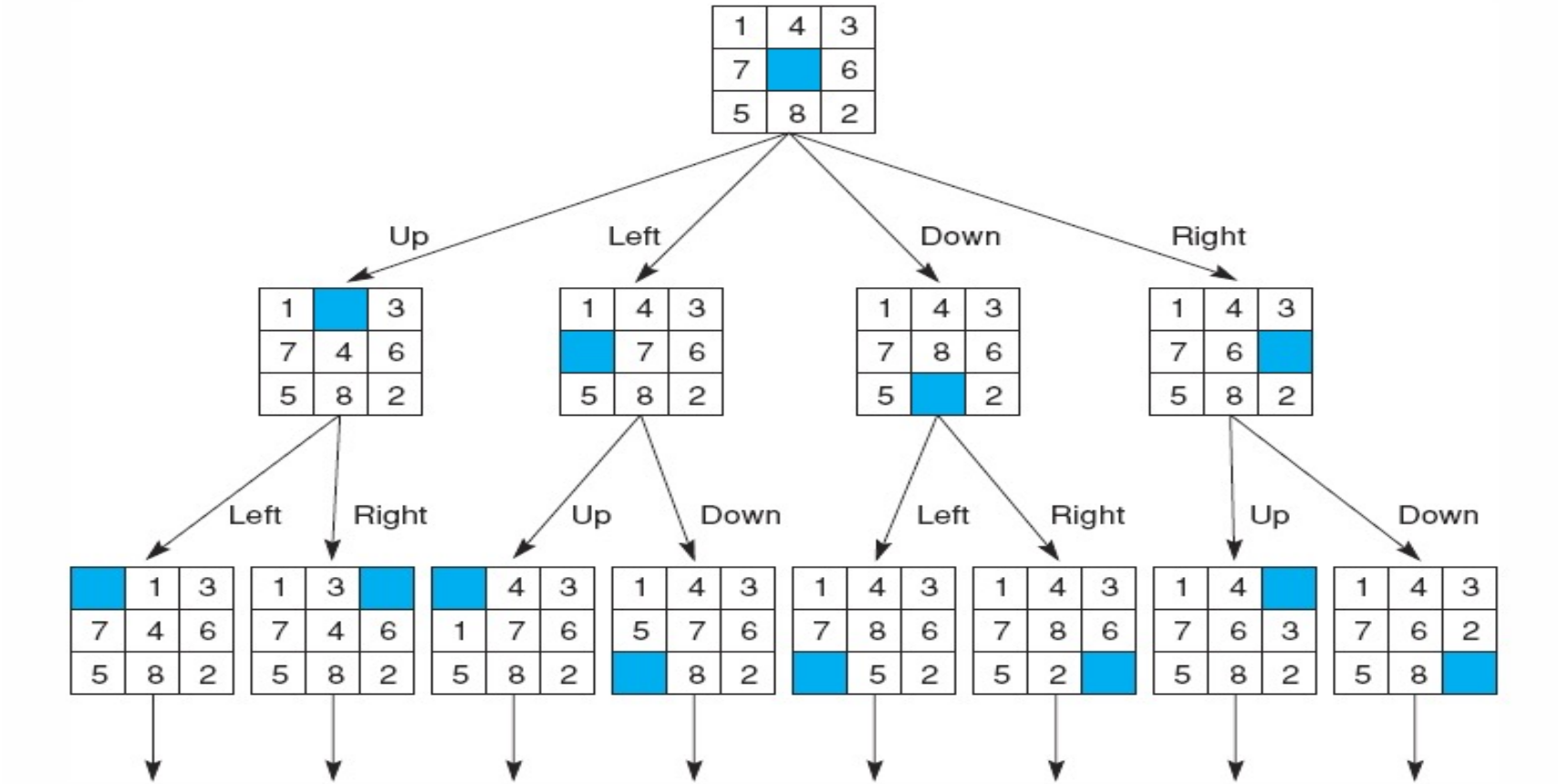
  - Es voluminoso
  - Difícil de caracterizar
  - Cambia
  - Comúnmente se organiza de la misma manera como se usará.

Técnicas para resolver problemas en IA

- Técnicas de IA en general, incorporan procesos de búsqueda.
- Los métodos de búsqueda pueden considerarse como árboles de búsqueda, en los que cada nodo representa un estado del problema y cada enlace representa la relación entre los estados vinculados.
- En general, hay reglas que determinan cómo navegar por el árbol, y partes del árbol pueden generarse o expandirse a medida que se activan las reglas.

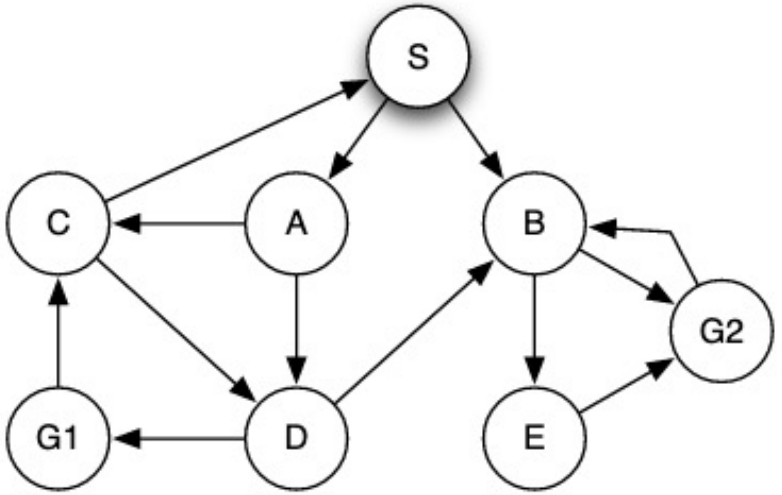


Técnicas para resolver problemas en IA



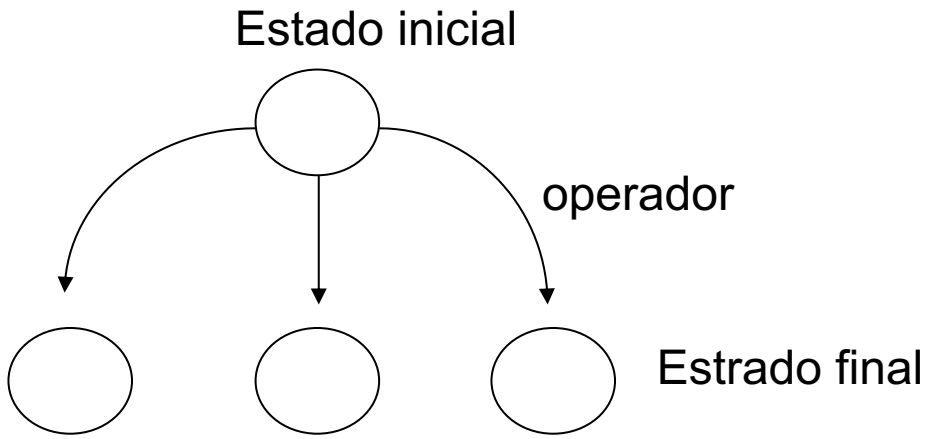
Técnicas para resolver problemas en IA

- Algunas veces el proceso de búsqueda puede involucrar la búsqueda en un grafo en lugar de un árbol, lo que puede ser beneficioso si exploramos rutas que son esencialmente las mismas. Sin embargo, hay algunos requisitos asociados:
  - Cada vez que se genera un nuevo nodo, debe verificarse si se ha creado en el pasado.
  - Necesita procedimientos especiales para evitar quedar atrapado en un lazo durante la búsqueda.



Técnicas para resolver problemas en IA

- Las reglas describen las acciones u operadores que permiten pasar de un estado a otro.
- Una regla está compuesta por un lado izquierdo y derecho
- El lado izquierdo determina la aplicabilidad de la regla; es decir, describa los estados a los que se pueden aplicar las reglas.
- El lado derecho describe la operación / acción que tiene lugar si se aplica la regla (se activa); es decir, pasar al siguiente estado.



Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Semestre 2021 – II

Agenda

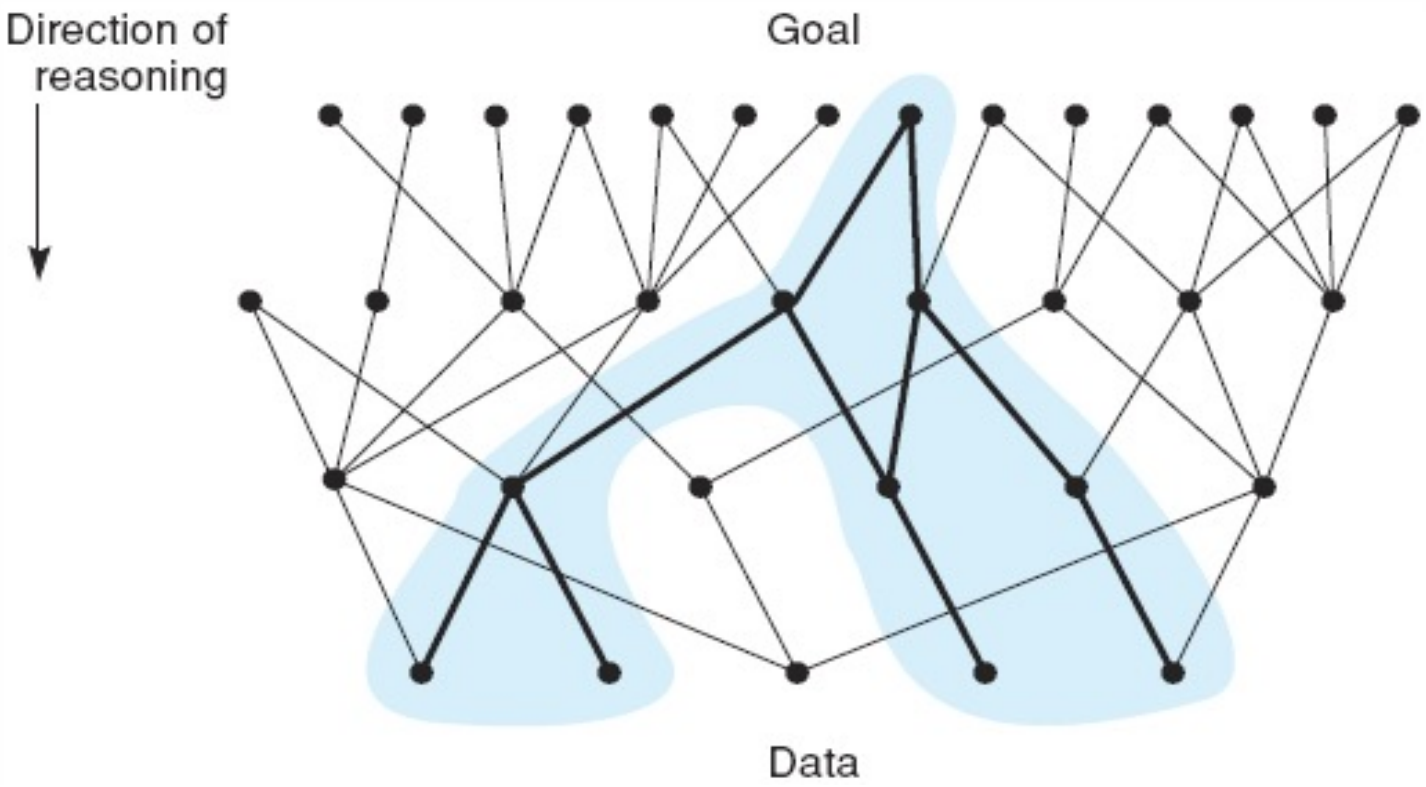
- Técnicas para resolver problemas en IA
  - Estrategias de control de búsqueda, conducida por datos y por objetivos
  - Algortimos de búsqueda no informada o a ciegas
  - Algoritmo Generar y Probar

Técnicas para resolver problemas en IA

- Estrategias para la búsqueda del espacio estatal
  - Goal-driven - encadenamiento hacia atrás
    - Toma el objetivo para ser resuelto. Ve qué reglas o movimientos legales podrían usarse para generar este objetivo y determina qué condiciones deben ser ciertas para usarlos.
    - Estas nuevas condiciones se convierten en los nuevos objetivos para la búsqueda, se remonta a los hechos del problema.
    - Esto encuentra la cadena de movimientos o reglas que conducen de los datos a la meta, pero al revés.

Técnicas para resolver problemas en IA

- Goal-driven

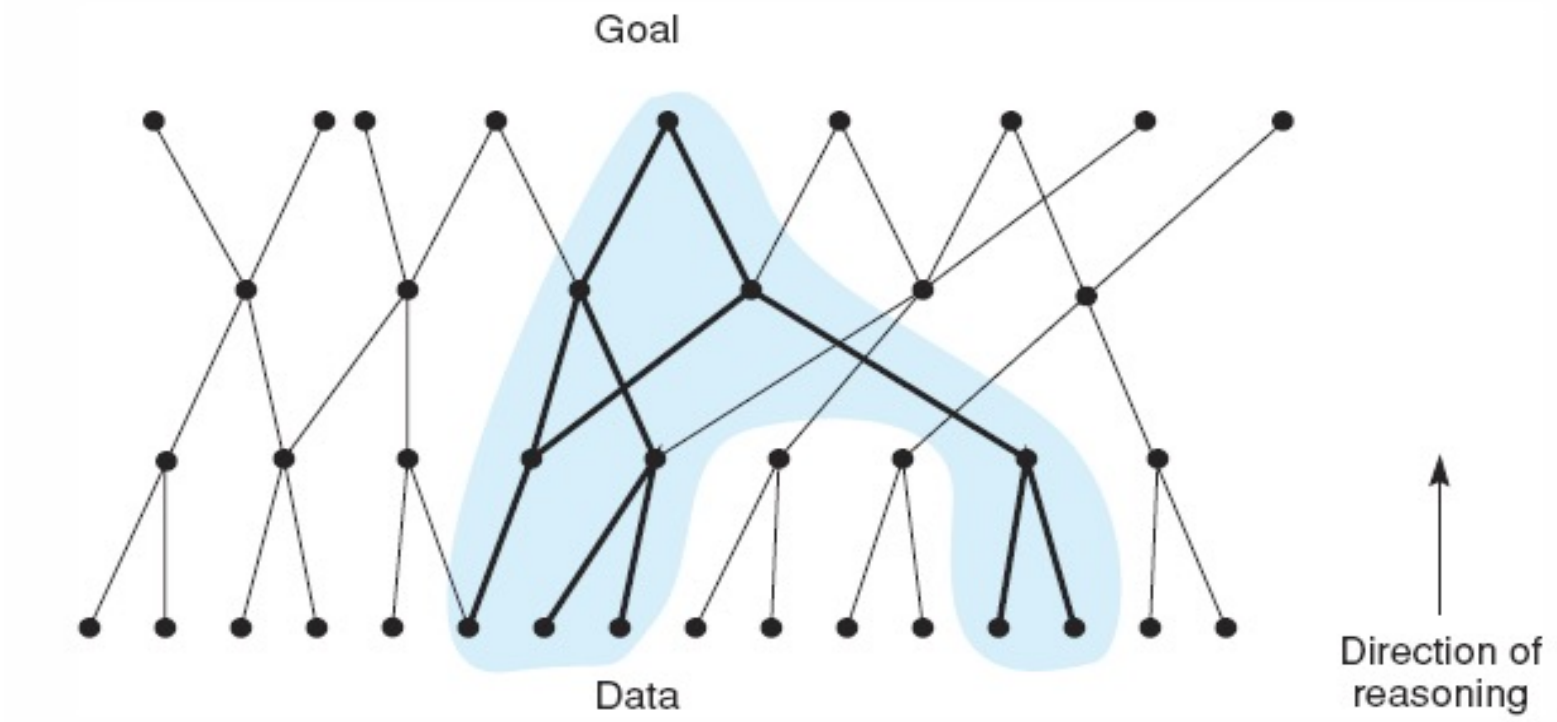


Técnicas para resolver problemas en IA

- Data-driven: encadenamiento hacia adelante
  - La búsqueda comienza con los hechos dados del problema y un conjunto de movimientos legales o reglas para cambiar el estado.
  - La búsqueda continúa aplicando reglas a los hechos para producir nuevos hechos, hasta que genera un camino que satisface la condición del objetivo.

Técnicas para resolver problemas en IA

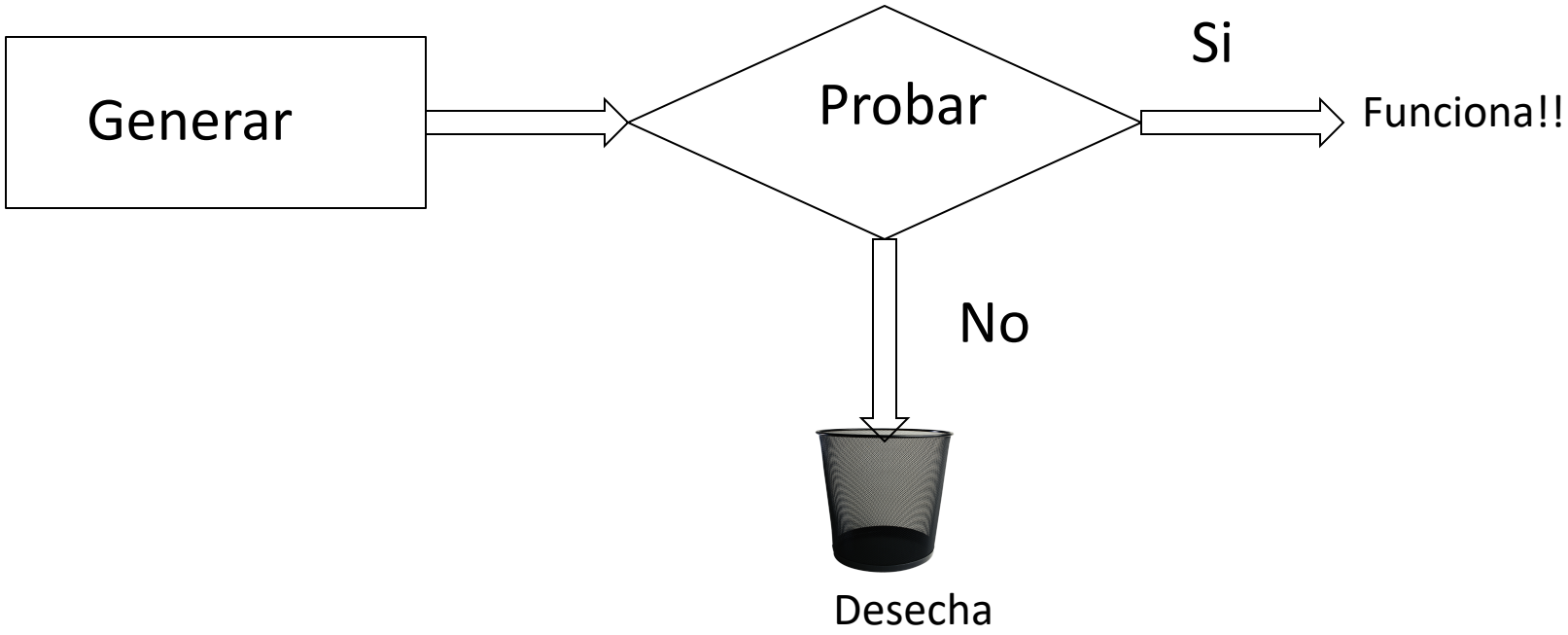
- Data Driven



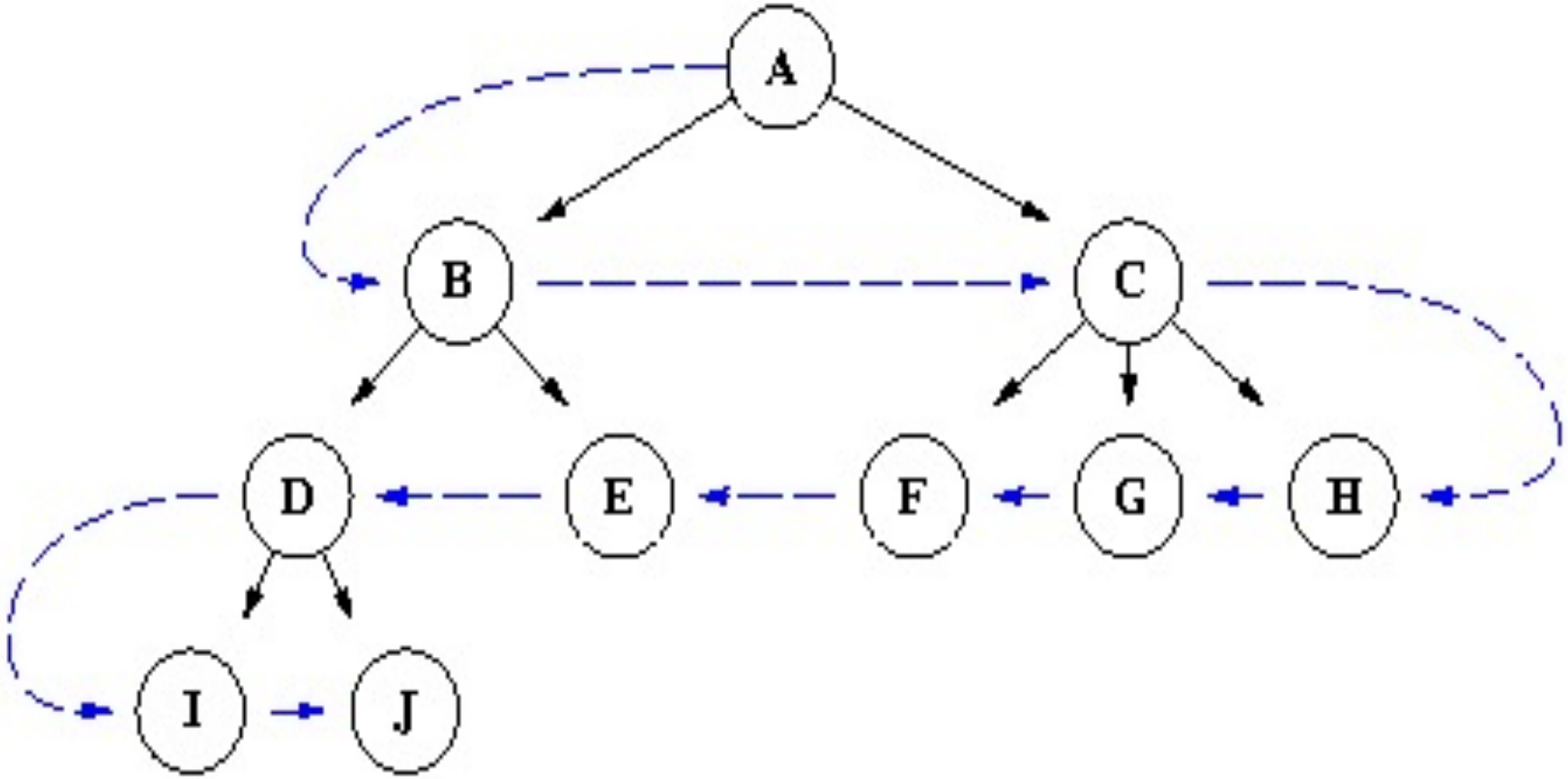
Técnicas para resolver problemas en IA

- Estrategias de búsqueda a ciegas:
    - Generate and Test
    - Breadth First
    - Depth First
    - Uniform cost search
    - Limited depth
    - Interactive depth
    - Bidirectional search

Generate and Test



Breath - First

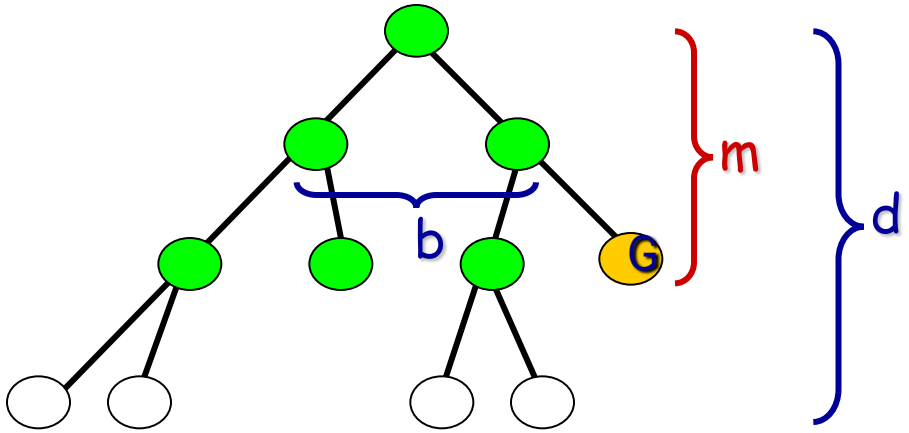


Breath - First

```
Begin
    open := [Start];
    closed := [ ];
    while open π [ ] do
        begin
            remove leftmost state from open, call it X;
                if X is a goal then returns SUCCESS
                    else begin
                        generate children of X;
                        put X on closed;
                        discard children of X if already on open or closed;
                        put remaining children on right end of open
                    end
        end
    return FAIL
end
```

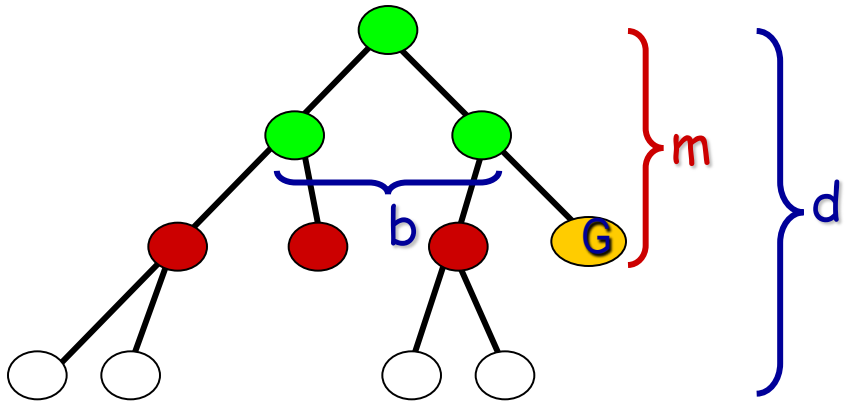
Breath - First

- Completo
  - ✓Si, (inclusive para arboles infinitos)
  - ✓Siempre encuentra la ruta más corta
- Tiempo
  - ✓$O(b^m)$

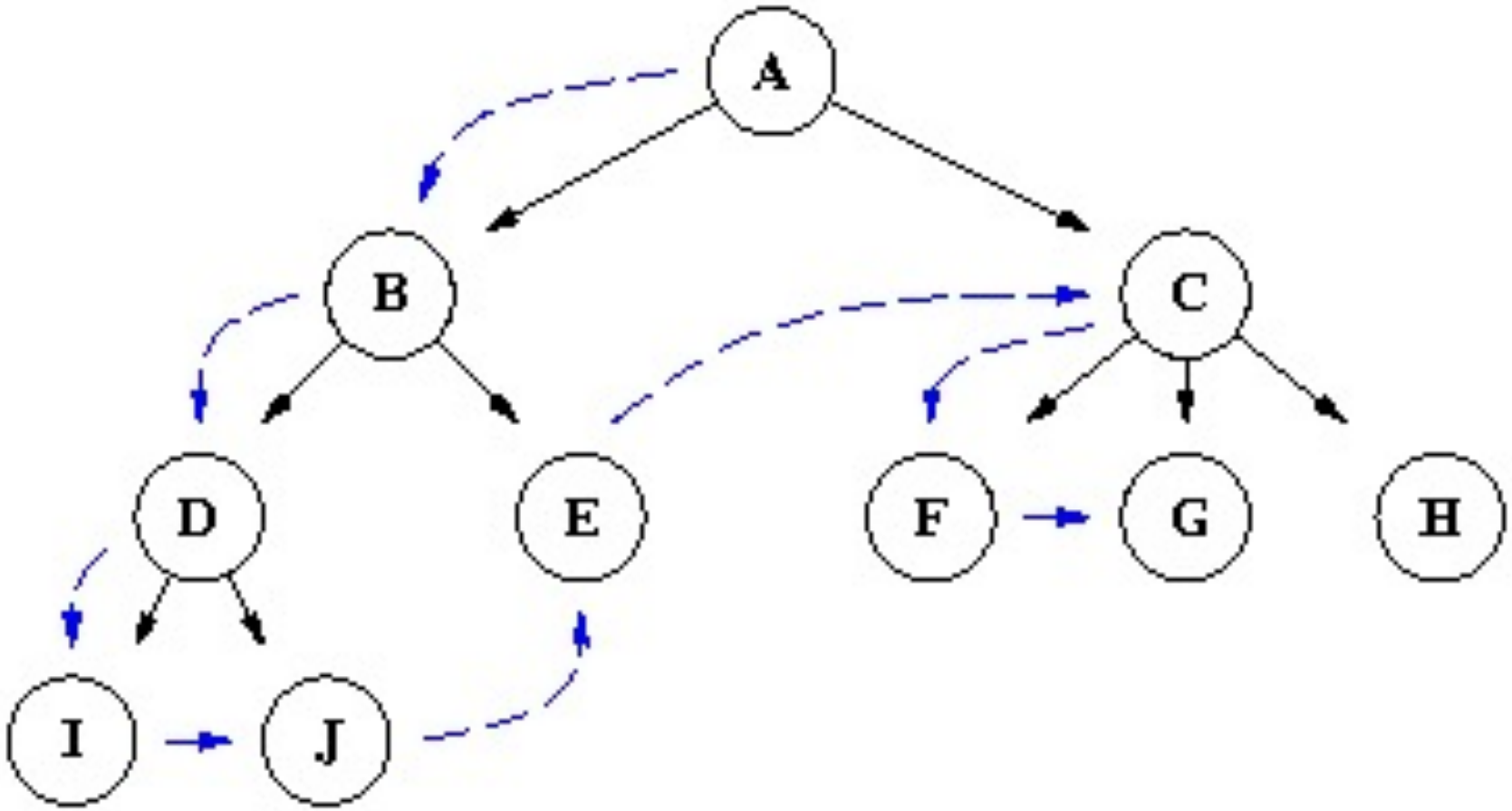


Breath- First

- Memoria
  - El número máximo de nodos se alcanza en el nivel "m" (objetivo)
  - En general: b^m
  - Generalmente "peor" que la profundidad primero



Depth- First

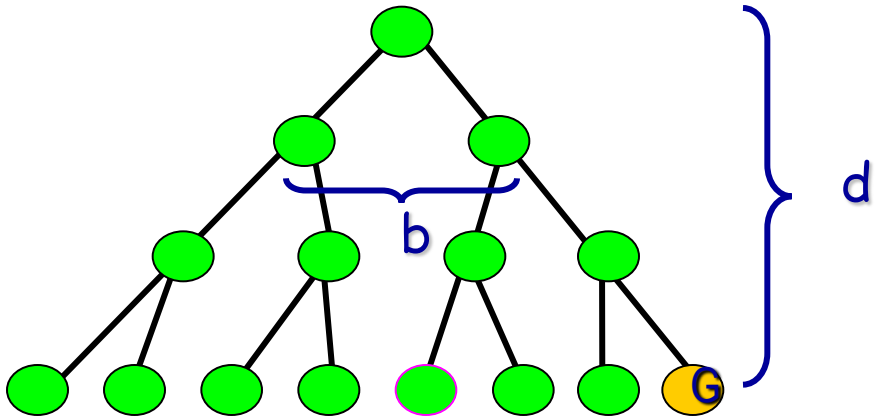


Depth- First

```
Begin
    open := [Start];
    closed := [ ];
    while open π [ ] do
        begin
            remove leftmost state from open, call it X;
                if X is a goal then returns SUCCESS
                    else begin
                        generate children of X;
                        put X on closed;
                        discard children of X if already on open or closed;
                        put remaining children on left end of open
                    end
        end
    return FAIL
end
```

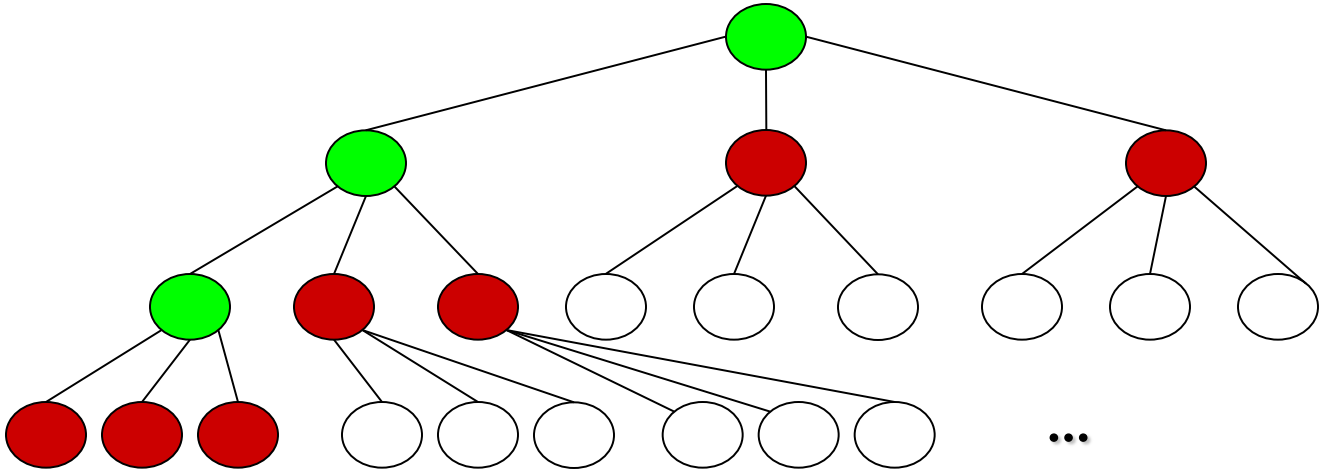
Depth- First

- Completo
  - Si -> árboles infinitos
- Tiempo
  - En el peor de los casos, el estado final podría ser la última rama del árbol.
  - $O(b^d)$



Depth- First

- Memoria
  - ✓En general: O(d*b)



Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Semestre 2021 – II

Agenda

- Búsqueda a ciegas
  - Generar y probar (Generate and Test)
  - Breath Search
  - Depth Search
- Introducción a la Búsqueda Heurística

Breath - First

```
Begin
    open := [Start];
    closed := [ ];
    while open π [ ] do
        begin
            remove leftmost state from open, call it X;
                if X is a goal then returns SUCCESS
                    else begin
                        generate children of X;
                        put X on closed;
                        discard children of X if already on open or closed;
                        put remaining children on right end of open
                    end
        end
    return FAIL
end
```

Depth - First

```
Begin
    open := [Start];
    closed := [ ];
    while open π [ ] do
        begin
            remove leftmost state from open, call it X;
                if X is a goal then returns SUCCESS
                    else begin
                        generate children of X;
                        put X on closed;
                        discard children of X if already on open or closed;
                        put remaining children on left end of open
                    end
        end
    return FAIL
end
```

Búsqueda heurística

- Búsqueda heurística (o búsqueda informada)
  - Del griego "heuriskein", que significa descubrimiento o hallazgo.
  - La heurística ayuda o proporciona información para guiar el proceso de búsqueda.
  - En general, con la heurística, no siempre encontraremos el resultado óptimo (la mejor solución), pero obtendremos muy buenos resultados en un tiempo razonable.
  - La heurística se usa para resolver problemas complejos donde generalmente tenemos una explosión combinatoria. En este tipo de problemas, los algoritmos de búsqueda exhaustivos tienen costos inaceptables (válidos solo con problemas simples).

Búsqueda heurística

- Utilizamos información heurística para "adivinar" qué nodo expandir o explorar a continuación.
- La información heurística aparece como una función, que se evalúa con base en información específica para el contexto o problema en cuestión.
- Por lo tanto, la solución del problema de búsqueda consiste en maximizar o minimizar una función.
- La función heurística nos permite evaluar un nodo "localmente", en función del costo estimado para alcanzar el objetivo desde el nodo actual.
- Hay algunos problemas con la heurística:
  - La heurística podría no ayudar a encontrar la mejor solución: problema abierto. Sin embargo, el algoritmo A * resuelve esto.
  - …

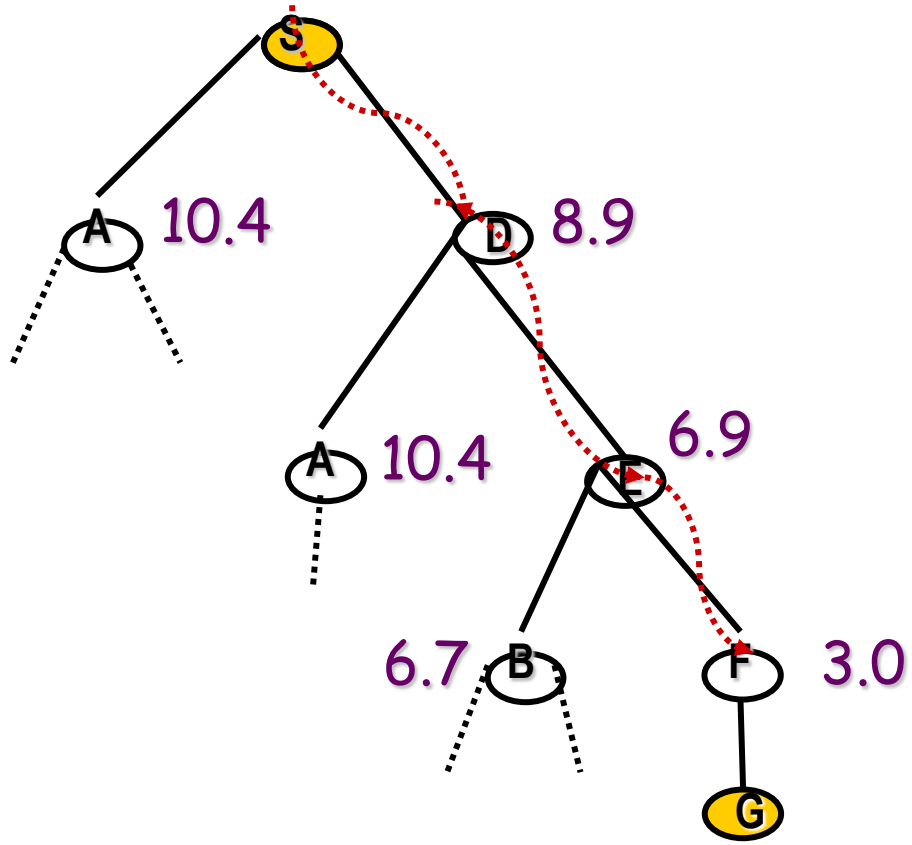
Búsqueda heurística

- La heurística es una técnica que siempre produce resultados, aunque a veces no son los mejores.
- Mucho de lo que hacemos en la vida real implica heurística; cuando buscamos soluciones o tomamos decisiones para problemas diarios.
- Por lo general, las soluciones funcionan o funcionan lo suficientemente bien; y, cuando no funcionan, tratamos el problema de una manera diferente.
- La heurística no solo se aplica a los casos en que una solución podría no estar disponible, sino también a los casos en los que queremos encontrar una mejor solución.

Búsqueda heurística

- Ejemplo: considere el problema de conducir entre ciudades: podríamos considerar cada ciudad como un nodo en un árbol, y las rutas entre ciudades, como un operador o regla, con alguna información heurística para ayudar al viajero a moverse de una ciudad a otra. otro.

Búsqueda heurística



- Es como la búsqueda primero a lo profundo, pero:
- En lugar de seleccionar el siguiente nodo de izquierda o derecha,
- Selecciona el nodo con el mejor valor heurístico.

Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Semestre 2021 – II

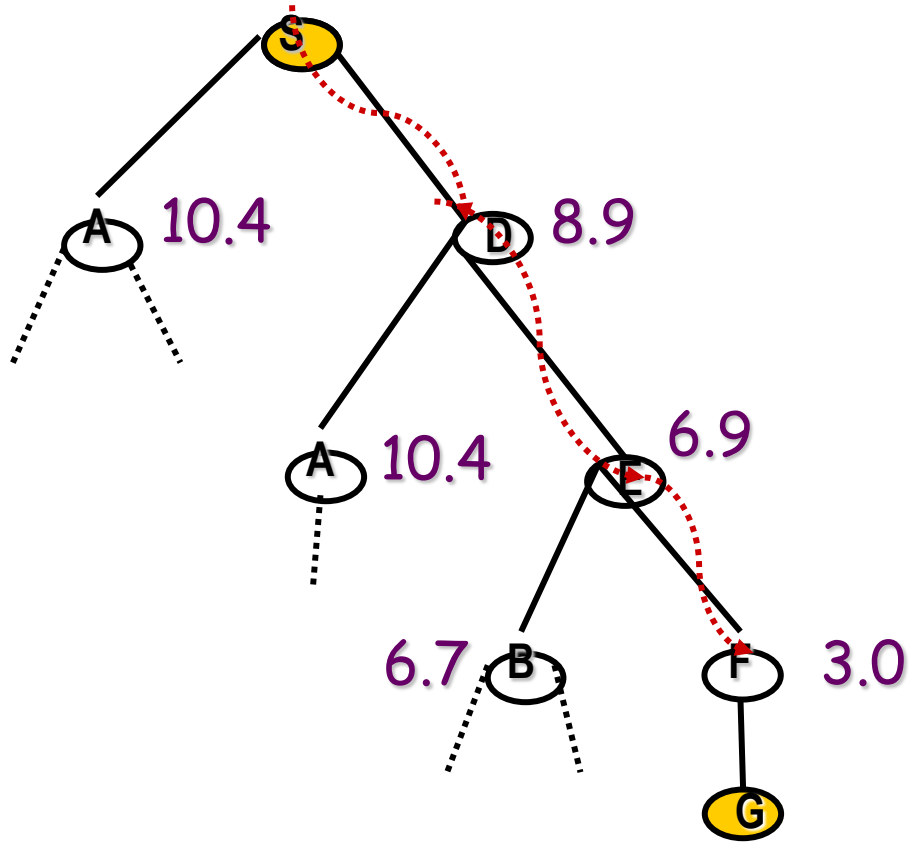
Agenda

- Búsqueda heurística
  - Hill Climbing
  - Simulated Annealing
  - Best – First

Búsqueda heurística

- Ejemplo: considere el problema de conducir entre ciudades: podríamos considerar cada ciudad como un nodo en un árbol, y las rutas entre ciudades, como un operador o regla, con alguna información heurística para ayudar al viajero a moverse de una ciudad a otra. otro.

Búsqueda heurística



- Es como la búsqueda primero a lo profundo, pero:
- En lugar de seleccionar el siguiente nodo de izquierda o derecha,
- Selecciona el nodo con el mejor valor heurístico.

Hill Climbing

- ***Hill Climbing***
  - Es una versión mejorada del algoritmo de generación y prueba.
  - Desde el procedimiento de prueba, hay una retroalimentación para ayudar al generador de la solución a decidir, en qué dirección ir a continuación y moverse en el espacio de la solución.
  - En este proceso, la búsqueda finaliza si no hay un próximo estado o nodo razonable para mover.

Simple Hill Climbing

1. Evaluar el estado inicial.
2. Repetir hasta que se encuentre una solución o no haya nuevos operadores para aplicar:

   - Seleccionar un nodo y aplicar la función heurística

   - Evaluar el nuevo estado:
3. objetivo $\rightarrow$ terminar
4. mejor que el estado actual $\rightarrow$ nuevo estado actual

Hill Climbing: Disadvantages

Maximo local

- Es el mejor estado o nodo en comparación con sus vecinos, pero no es el mejor entre otros, que podría estar adelante en el espacio de la solución.

Hill Climbing: Desventajas

Meseta

- Un área plana del espacio de búsqueda en el que todos los estados vecinos tienen el mismo valor.

Hill Climbing: Desventajas

Acantilado

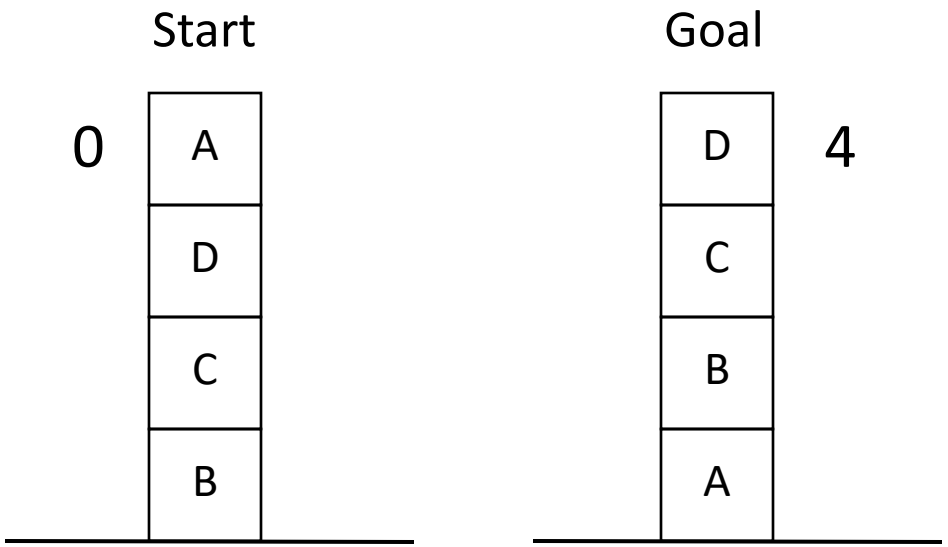
- Es un tipo especial de máximo local, que es imposible de cruzar con movimientos simples.

Hill Climbing: Desventajas

Salidas

- Retroceder a algún nodo anterior e intentar ir en una dirección diferente.
- Dar un gran salto para intentar ingresar a una nueva sección del espacio de solución.
- Moverse en varias direcciones a la vez.

Hill Climbing: Meseta

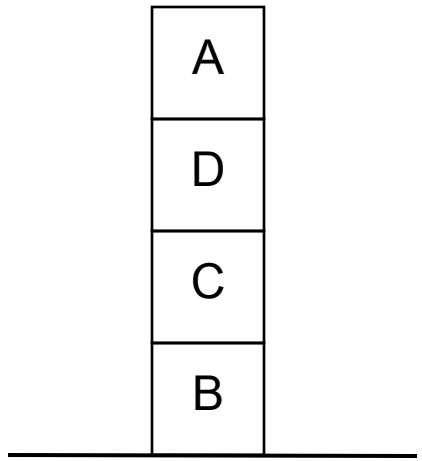


Blocks World

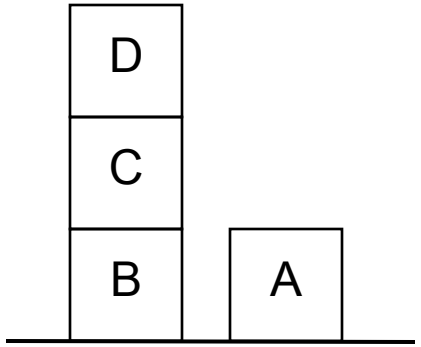
Local heuristic – plateau problem:

+1 for each block that is located on the position it is supposed to be located on.
-1 for each block that is located in a wrong position.

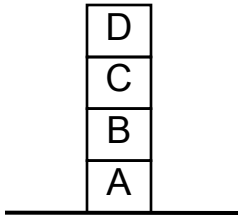
Hill Climbing: Meseta

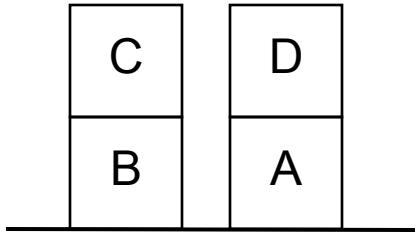


0

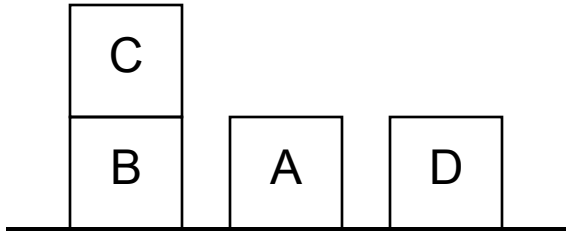


2





1-1+1-1=0



1-1+1-1=0

Simulated Annealing

- Es una versión mejorada del algoritmo Hill Climbing.
- Al principio, este algoritmo explora una buena parte del espacio de la solución, de tal manera que la solución final podría no verse afectada por el estado inicial.
- Como consecuencia, la posibilidad de quedar atrapado en un máximo local, mesa o acantilado, es pequeña.

Simulated Annealing

1. Evaluar el estado inicial.
2. Repetir hasta que se encuentre una solución o no haya nuevos operadores para aplicar:

   - Establecer T según un estado inicial

   - Selecciona y aplica un nuevo operador

   - Evaluar el nuevo estado:

     goal $\rightarrow$ terminar

     ΔE = Valor (estado actual) - Valor (nuevo estado)

     $\Delta E < 0 \rightarrow$ nuevo estado actual

     else $\rightarrow$ nuevo estado actual con probabilidad $e^{-\Delta E/kT}$

Best-First Search

- **Best - First**
  1. **Lista open**: contiene los nodos generados.

     A los nodos se les ha aplicado la función heurística; pero aún no han sido probados; es decir, sus sucesores aún no se han generado. Esta cola puede considerarse como una cola de prioridad, en la que los elementos con mayor prioridad son los que tienen valores más altos, asignados por la función heurística.
  1. **Lista close**: contiene los nodos que se han examinado y no son el objetivo esperado.
  2. **Función heurística**: permite que el algoritmo busque primero las rutas más prometedoras.

Best-First Search

Función heuristica:

$$f(n) = g(n) + h(n)$$

- g(n) = costo de la ruta más barata desde el estado inicial al nodo n.
- h(n) = costo de la ruta más barata desde el nodo n a un estado objetivo.

Best-First Search

- Greedy search:

  h(n) = costo estimado de la ruta más barata desde el nodo n hasta un estado objetivo.

  Ni óptimo ni completo

Best-First Search

- Uniform-cost search:

  g(n) = costo de la ruta más barata desde el estado inicial hasta el nodo n.

  Óptimo y completo, pero muy ineficiente

Best-First Search

- Algoritmo A

$$f(n) = g(n) + h(n)$$

Best-First Search

- **Best - First**
  - La función heurística ***f*** se puede definir como la suma de otras dos funciones, llamadas ***g*** y ***h***.
  - La función ***g*** es el costo para alcanzar el estado actual desde el estado inicial.
  - La función ***h*** es el costo estimado para alcanzar el estado objetivo desde el estado actual.

$$f(n) = g(n) + h(n)$$

Best-First Search

```
function best_first_search;

begin
   open := [Start];                                              % initialize
   closed := [ ];
   while open ≠ [ ] do                                           % states remain
      begin
         remove the leftmost state from open, call it X;
         if X = goal then return the path from Start to X
         else begin
                  generate children of X;
                  for each child of X do
                  case
                        the child is not on open or closed:
                           begin
                                 assign the child a heuristic value;
                                 add the child to open
                           end;
                        the child is already on open:
                           if the child was reached by a shorter path
                           then give the state on open the shorter path
                        the child is already on closed:
                           if the child was reached by a shorter path then
                              begin
                                 remove the state from closed;
                                 add the child to open
                              end;
                  end;                                           % case
                  put X on closed;
                  re-order states on open by heuristic merit (best leftmost)
               end;
return FAIL                                                      % open is empty
end.
```

Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Semestre 2021 – II

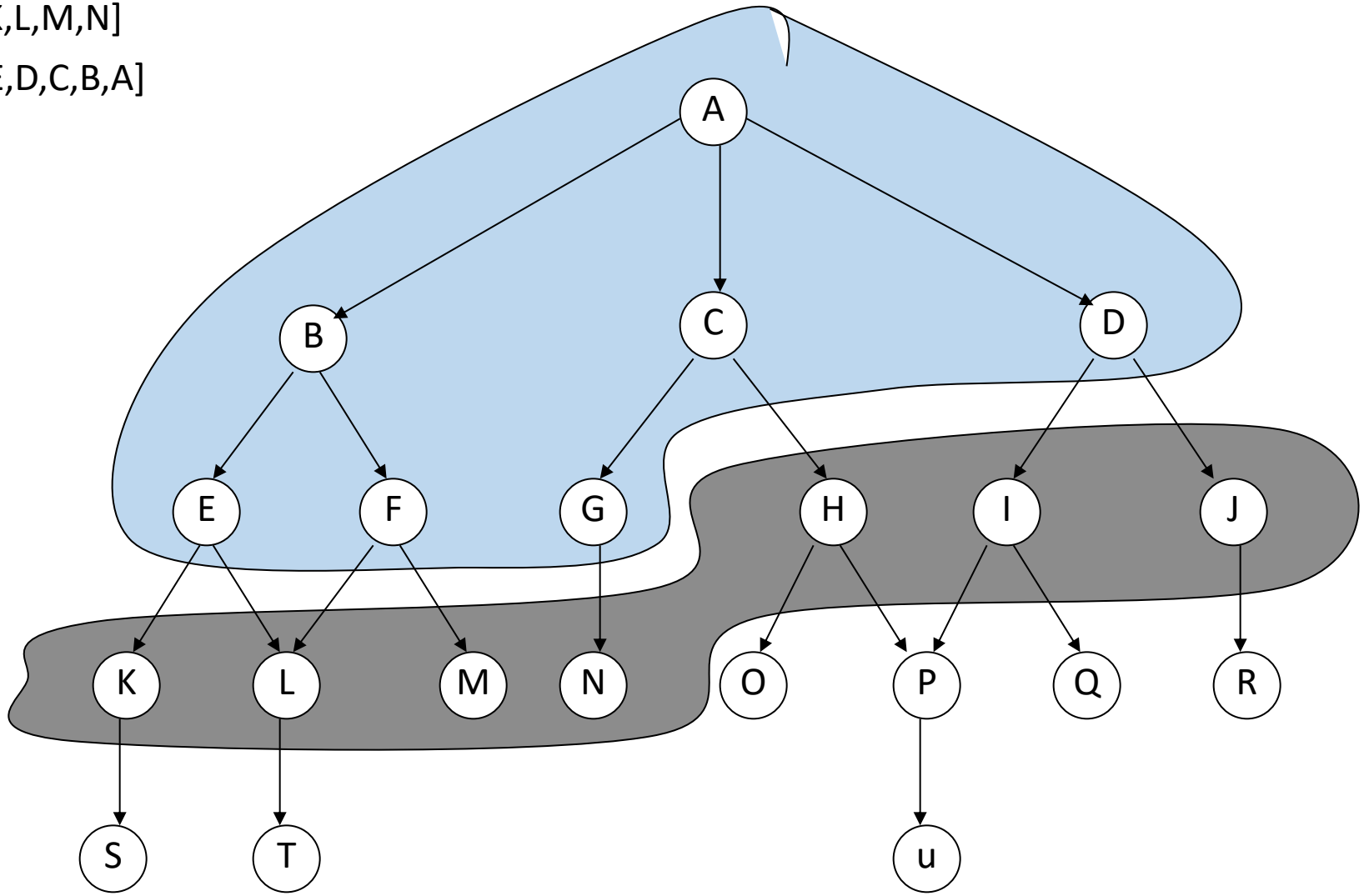
Agenda

- Fronteras
- Teorema de Admisibilidad
- Algoritmo Backtrack

Breadth- First

open = [H,I,J,K,L,M,N]

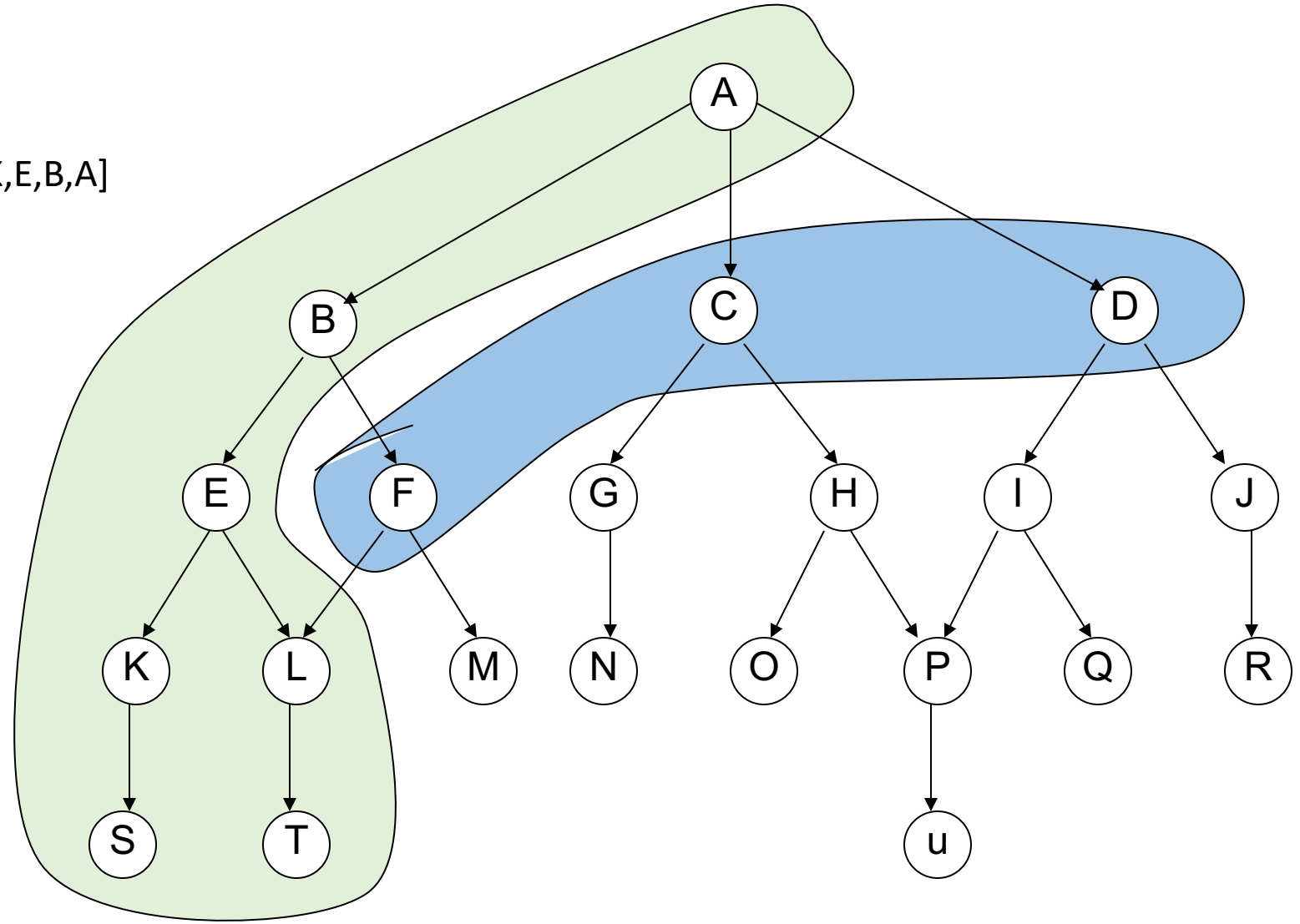
closed = [G,F,E,D,C,B,A]



Depth - First

open = [F,C,D]

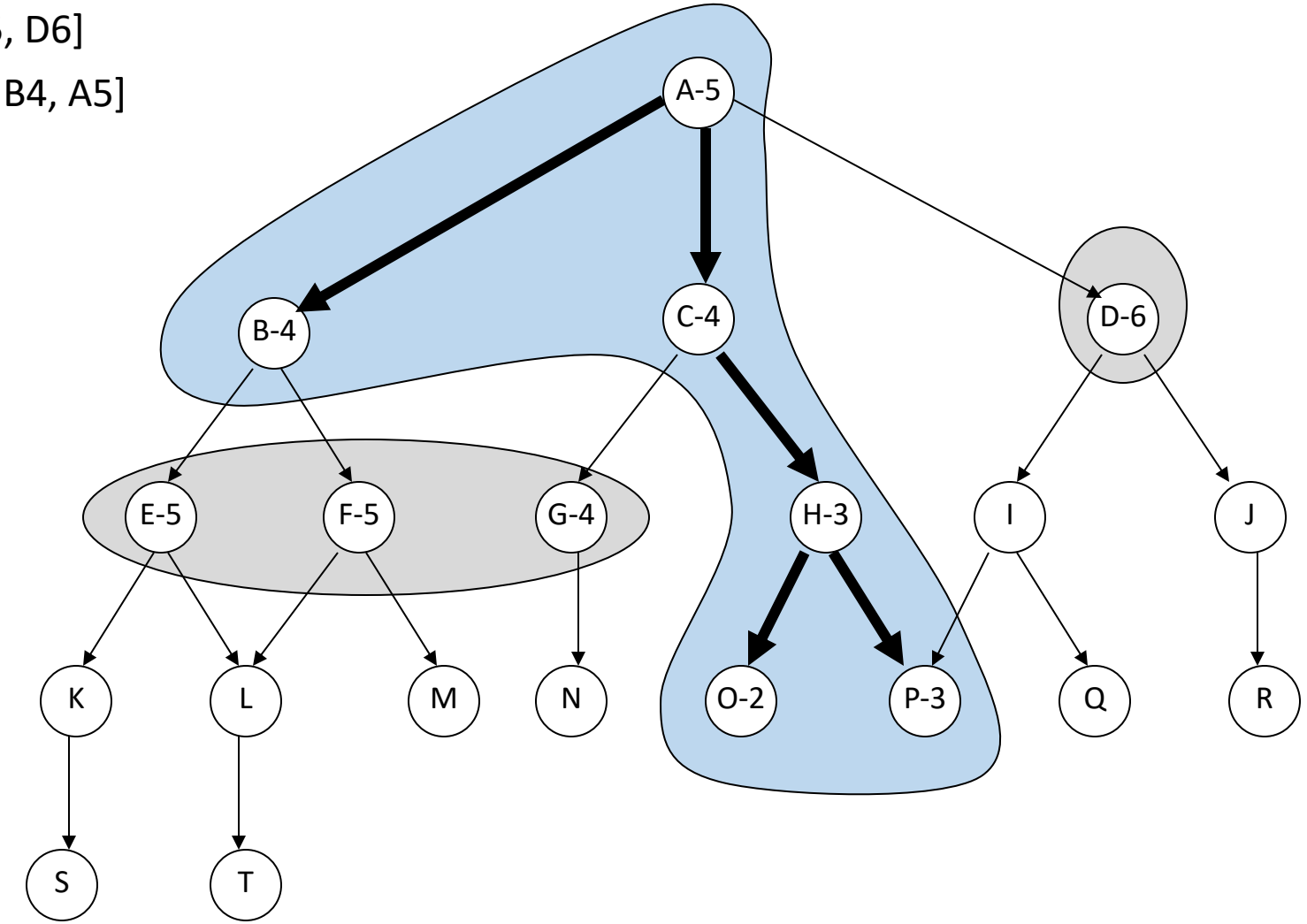
closed = [T,L,S,K,E,B,A]



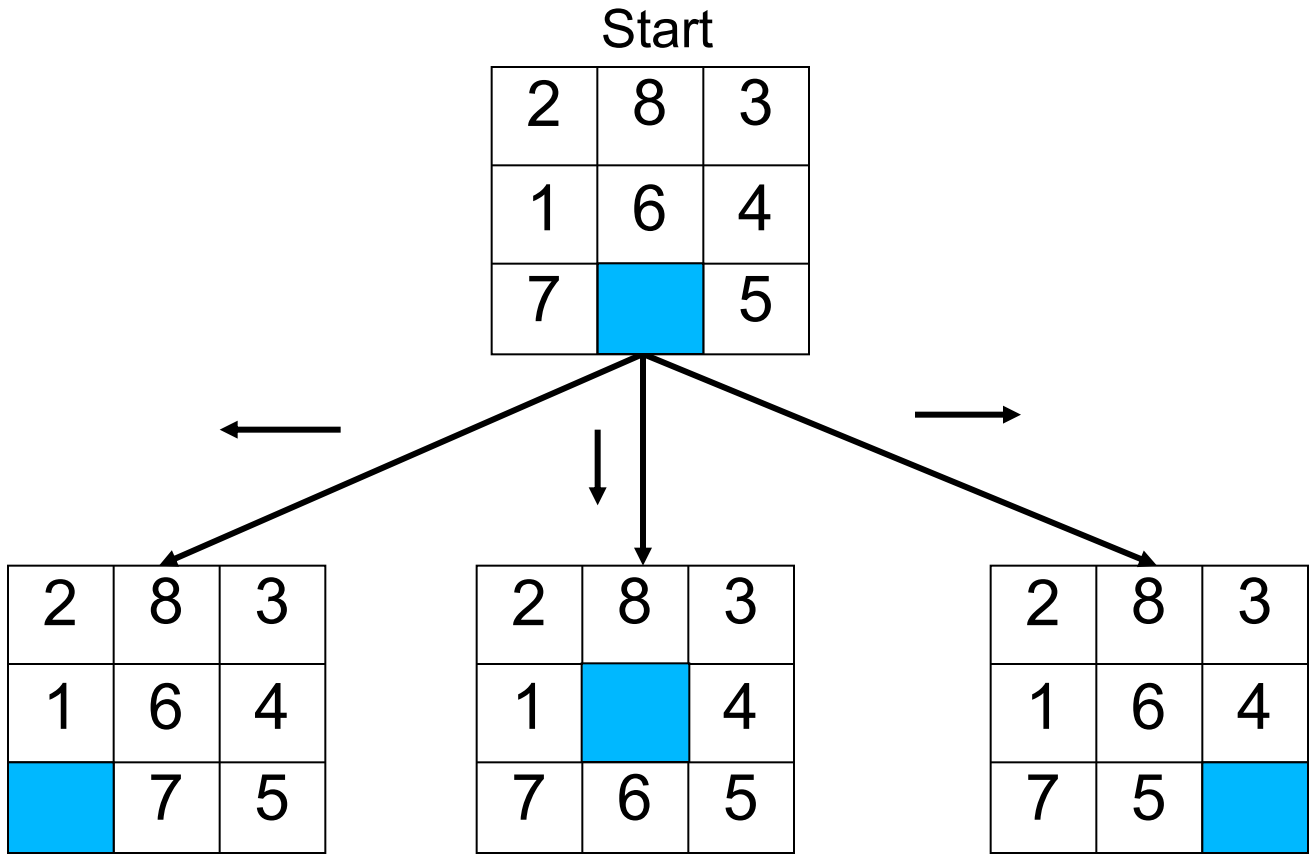
Best- First

open = [G4, E5, F5, D6]

close [O2, H3, C4, B4, A5]



Búsqueda heurística



Goal

1	2	3
8		4
7	6	5

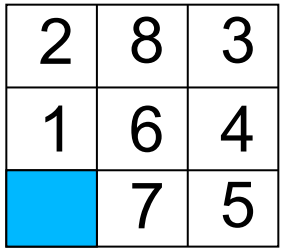
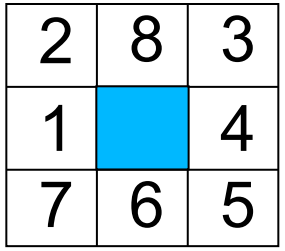
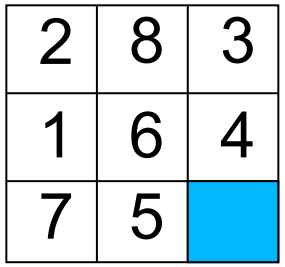
Búsqueda heurística

Start

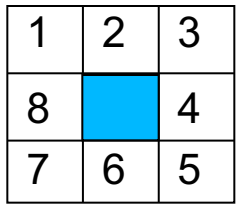
2	8	3
1	6	4
7		5

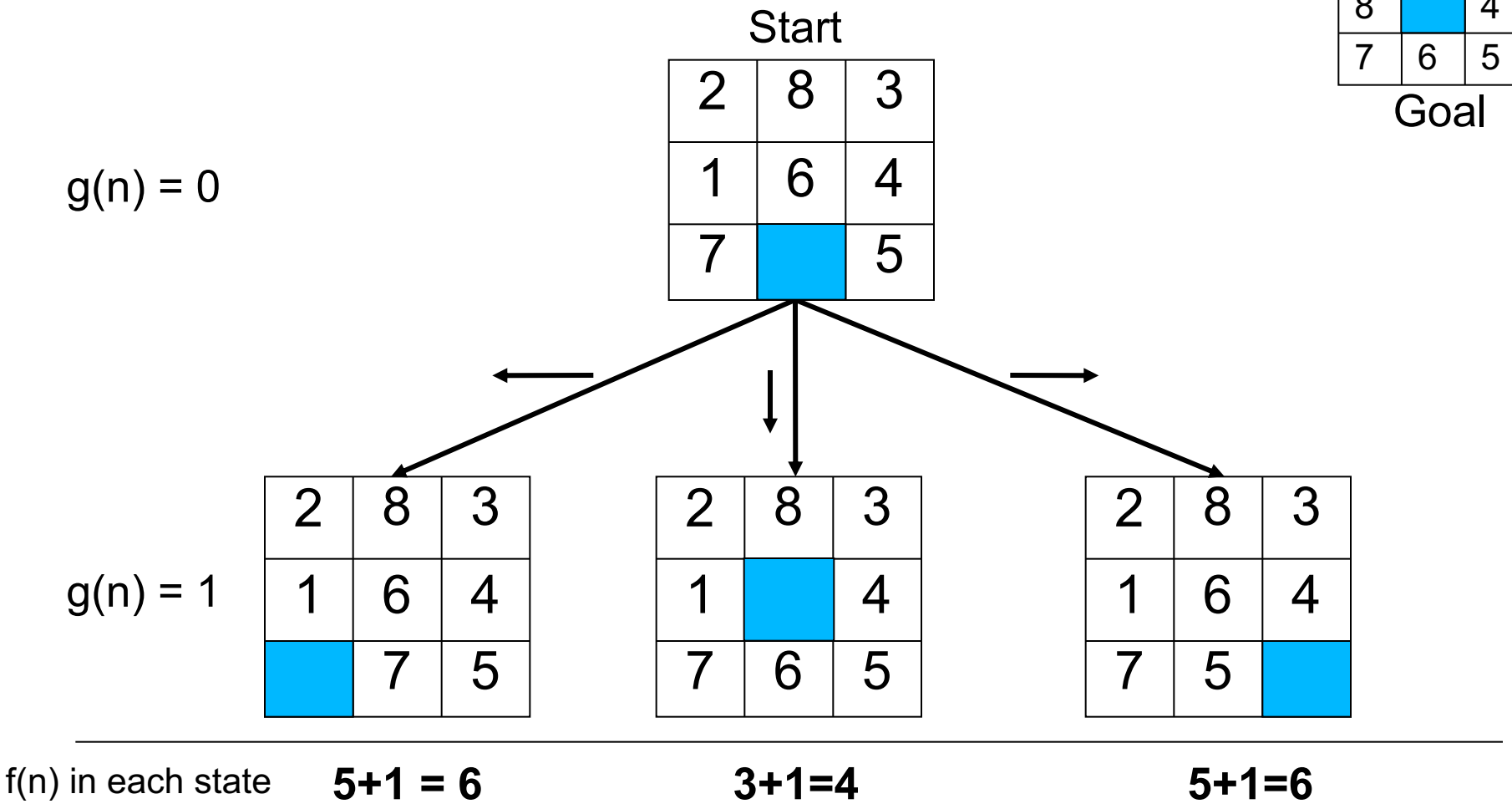
Goal

1	2	3
8		4
7	6	5

	5	6	0
	3	4	0
	5	6	0
h(n)	Tiles out of place	Sum of distances of the tiles out of place	2 x # of reversed tiles

Búsqueda heurística





Búsqueda heurística

- Considere la función: f (n) = g (n) + h (n)

Dónde:

- *n* es cualquier estado encontrado durante la búsqueda
- *g* es el costo desde el estado inicial hasta el estado actual *n*
- *h* es el valor heurístico o el costo adicional para alcanzar la meta desde el estado actual *n*

- Si esta función se usa con el algoritmo de búsqueda primero el mejor, entonces se llama algoritmo tipo A.
- Cuando un algoritmo heurístico garantiza encontrar una solución óptima, si existe, entonces el algoritmo se llama Admisible, A.

Búsqueda heurística

Considerar:

$$f^*(n) = g^*(n) + h^*(n)$$

Dónde:

- $g^*(n)$ = costo de la ruta más corta desde el estado inicial hasta n.
- $h^*(n)$ = costo de la ruta más corta desde n hasta la meta.

- Si se usa un algoritmo A con una función de evaluación f^*, donde $h^*(n) \leq h(n)$, el algoritmo se llama A *

Búsqueda heurística

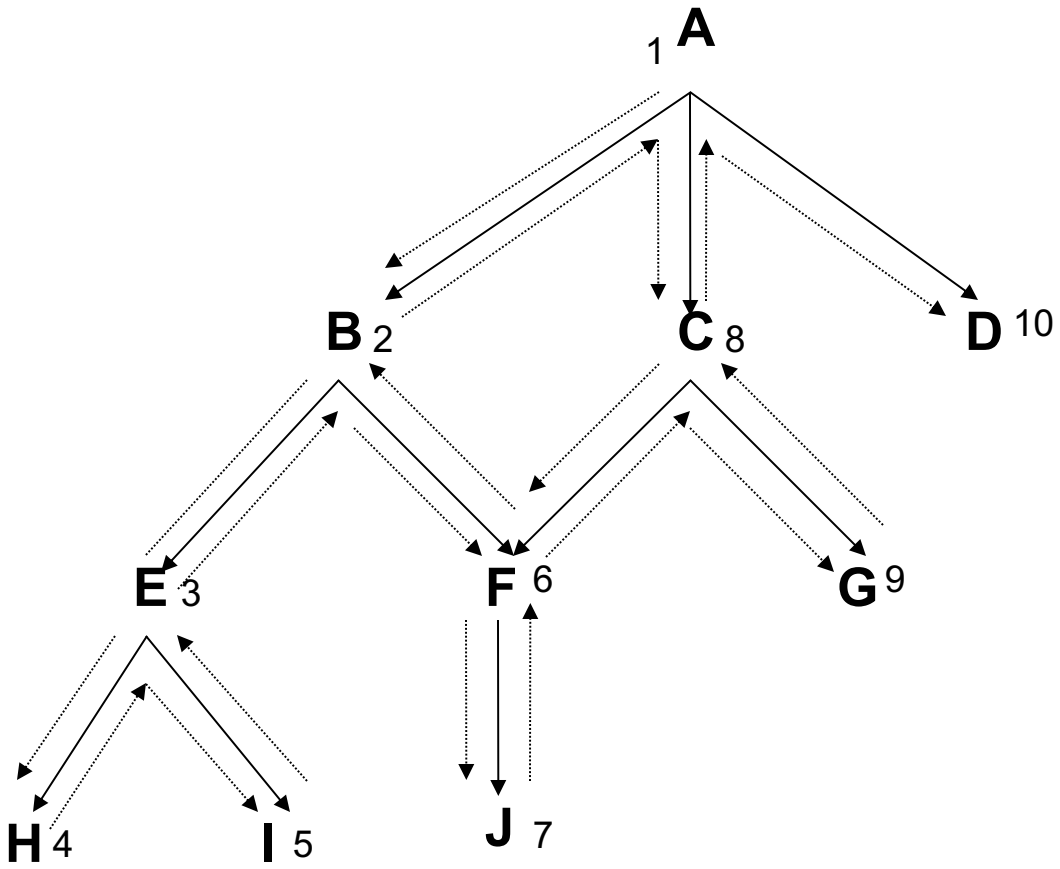
- **A* Teorema de Admisibilidad**

  - Un algoritmo A *, que usa h (n), donde h *(n) ≤ h (n), para todo n, entonces el algoritmo garantiza encontrar una ruta óptima desde n hasta el objetivo, si esta ruta existe.
  - Primero a lo ancho se puede clasificar como un algoritmo A *, en el que f (n) = g (n) + 0

Backtrack algorithm

```
begin
    SL:=[start]; NSL:=[start]; DE:=[ ]; CS:= start;
    While NSL π [ ] do
        begin
            if CS:= goal (or meets the goal description)
                then return SL;
            if CS has no children (excluding the nodes already in DE, SL and NSL)
                then
                    begin
                        while SL not empty and CS = first element of SL do
                            begin
                                add CS to DE;
                                remove first element of SL;
                                remove first element of NSL;
                                CS := first element of NSL;
                            end
                        add CS a SL;
                    end
                else
                    begin
                        place the children of CS (except if already in DE, SL or NSL) in NSL;
                        CS := first element of NSL;
                        add CS a SL
                    end
        end;
    return FAIL;
end.
```

Backtrack algorithm



Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Semestre 2021 – II

Agenda

1. Representación del conocimiento
    - Representación lógica matemática

Representación del conocimiento

- El método debe ser:
  - Expresivo
    - Permitir representar toda la información necesaria.
    - Proporcionar un esquema natural para representar el conocimiento requerido.
  - Eficiente
    - Para apoyar una ejecución eficiente de código de computadora.
    - En general para permitir resolver no solo problemas cuantitativos, sino también cualitativos.
    - Organizar grandes y diferentes tipos de conocimiento, y permitir implementar algoritmos de búsqueda.

Representación del conocimiento

- El método de representación del conocimiento debería permitir:
  - Inferir nuevos conocimientos a partir de hechos básicos.
  - Representar principios generales, así como situaciones específicas.
  - Capture el significado semántico complejo.
  - Representa el metaconocimiento.

Representación del conocimiento

- Esquemas de representación del conocimiento:

1. Representación lógica:
   - Utiliza expresiones lógicas formales para representar el conocimiento.
   - Cálculo del predicado de primer orden.

2. Representación procesal:
   - El conocimiento se representa como un conjunto de instrucciones.
   - Regalas tipo: SI - ENTONCES.

Representación del conocimiento

3. Representación de red:

   El conocimiento se captura en forma de gráfico.

   Los nodos representan objetos o conceptos, y los enlaces representan relaciones o asociaciones entre ellos.

   Redes semánticas, gráficos conceptuales, mapas cognitivos.

4. Representación estructurada:

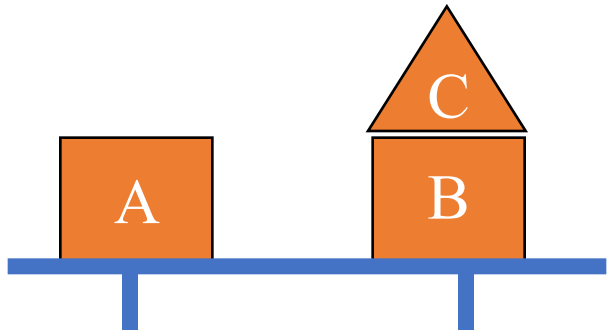
   Esta es una extensión de la representación de la red.

   Los nodos son estructuras complejas que contienen "ranuras con valores asociados"

   Scripts, marcos, objetos.

Representación del conocimiento y lógica de predicados

- La representación del conocimiento debe permitir capturar y razonar sobre los aspectos cualitativos de un problema.



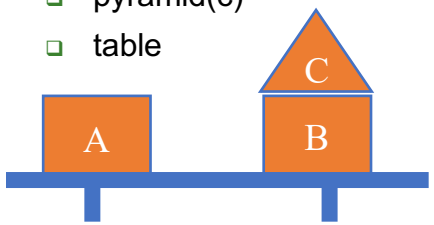
- free(c), free(a)
- on_table(a), on_table(b)
- above(c, b)
- cube(a), cube(b)
- pyramid(c)
- table

Representación del conocimiento y lógica de predicados

- En IA es esencial inferir nuevos conocimientos adicionales basados en conceptos básicos capturados.
  - No es posible describir todas las situaciones posibles; pero, necesitamos conocer las estrategias y reglas y cómo aplicarlas.
  - Ejemplo: Cuál sería una regla general para inferir que un bloque es libre ?:

$$\forall X \neg \exists Y \text{ above}(Y, X) \Rightarrow \text{free}(X)$$

- free(c), free(a)
- on_table(a), on_table(b)
- above(c, b)
- cube(a), cube(b)
- pyramid(c)
- table



Representación del conocimiento y lógica de predicados

- Cálculo proposicional
    - Las palabras, frases o declaraciones (proposiciones) en el cálculo proposicional y predicados, se pueden usar para representar y razonar sobre las propiedades y relaciones de cualquier objeto en el mundo real.

Representación del conocimiento y lógica de predicados

- Semántica en cálculo proposicional

  - El razonamiento con cálculo proposicional, en sistemas inteligentes, se realiza a través de los símbolos y estructuras utilizados para representar el conocimiento.
  - La verdad de las conclusiones depende solo de la verdad del conocimiento inicial.
  - Los errores lógicos no son introducidos por los procedimientos de inferencia artificial.
  - La interpretación de las proposiciones se realiza mediante la verdad valorada asignada (T o F) a cada símbolo proposicional.

Representación del conocimiento y lógica de predicados

Predicados

- El cálculo predicado proporciona los medios para acceder a los componentes individuales de una proposición:

  Ejemplo: Si el símbolo P, representa la siguiente proposición:

  "Hoy está nublado"

- Con el clima predicado podemos describir la relación entre la fecha y el clima.

  clima (hoy, nublado)

Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

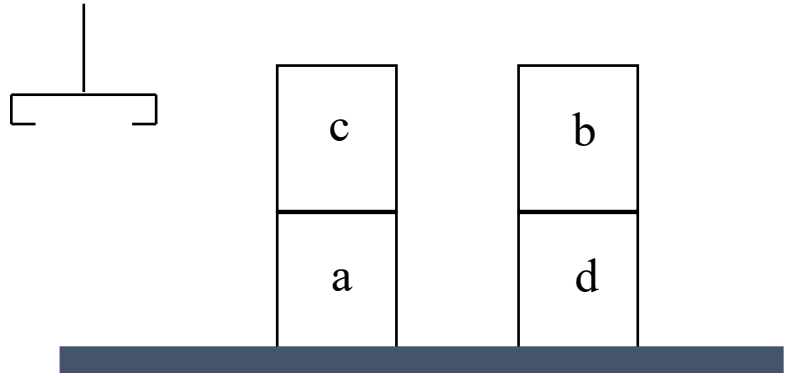
Semestre 2021 – II

Agenda

1. Representación del conocimiento
    - Sintaxis
    - Reglas de inferencia artificial

Representación del conocimiento y lógica de predicados

- above (*c, a*)
- above (*b, d*)
- on_table (*a*)
- on_table (*d*)
- free (*b*)
- free (*c*)
- arm_free ()
- table ()



Representación del conocimiento y lógica de predicados

- Supongamos que la computadora conoce la ubicación de cada bloque y el brazo del robot; y recuerda sus coordenadas.
- Para tomar un bloque y apilarlo sobre otro, los dos bloques deben estar libres.
- Inferir que un bloque es libre:

$$\text{bloque libre: } \forall X (\neg \exists Y \text{ above } (Y, X) \Rightarrow \text{free } (X))$$

- Para apilar un bloque X sobre Y, primero libere el brazo, libere X, luego libere Y, tome X y apílelo sobre Y.

$$\forall X \forall Y (\text{arm_free } () \wedge \text{free } (X) \wedge \text{free } (Y) \wedge \text{take } (X) \wedge \text{pile_over } (X, Y)) \Rightarrow \text{pile_up } (X, Y))$$

Métodos de razonamiento: Reglas de inferencia

Reglas de inferencia

- La semántica en el cálculo de predicados proporciona las bases para una teoría de inferencia formal en la lógica matemática.
- La capacidad de inferir nuevas expresiones a partir de un conjunto de afirmaciones verdaderas es la clave en el cálculo de predicados.
- Las nuevas expresiones son **correctas** porque son consistentes con las interpretaciones originales.
- Cuando una interpretación hace que una afirmación sea verdadera, se dice que la interpretación **satisface** la afirmación.

Métodos de razonamiento: Reglas de inferencia

- Una regla de inferencia en el cálculo de predicados es esencialmente un mecanismo para producir nuevas aserciones a partir de otras aserciones.
- Cuando una regla de inferencia ***R*** se aplica a un conjunto ***S*** de expresiones lógicas y produce nuevas aserciones ***X***, la regla de inferencia es **sólida y correcta**.
- Si la regla de inferencia tiene la capacidad de producir ***nuevas expresiones / reglas*** que sigan lógicamente el conjunto ***S***, entonces se dice que la regla está **completa**.

Ejemplo: la regla de resolución

Métodos de razonamiento: Unificación

Unificación

- Para inferir una nueva regla a partir de un conjunto de reglas, debemos determinar si las expresiones son equivalentes.
- Dos expresiones son equivalentes si y solo si son sintácticamente iguales.
- La unificación es un algoritmo que permite determinar las sustituciones necesarias para hacer equivalentes dos expresiones.

Métodos de razonamiento: Unificación

- Para permitir la libertad en las sustituciones, todas las variables deben cuantificarse universalmente.
- Las variables cuantificadas existencialmente pueden eliminarse reemplazándolas con valores constantes que hacen que la expresión sea verdadera.

  $\forall$ X $\exists$ Y mama (Y, X)

  Y depende del valor de X
- Skolemization es una acción que reemplaza cada variable cuantificada existencialmente con una función que devuelve el valor constante apropiado del rango, en función de otra variable.

Métodos de razonamiento: Unificación

- En nuestro ejemplo, Y puede ser reemplazado por una función skolem *f* (X).

  $\forall$ X mama (*f* (X), X).

- La notación Y / X indica que Y ha sido reemplazado por la variable X, en la expresión original.
- Una variable no se puede unificar con un término que contiene la misma variable.

Ejemplo: X no se puede reemplazar por g (X), porque produce la expresión: g(g(g(g(...X)...)

Métodos de razonamiento: Unificación

- Si una variable X1 se reemplaza por otra variable X2, y luego se reemplaza por una constante, entonces X1 debe reflejar ese último cambio.
- El conjunto completo de sustituciones utilizado en el proceso de inferencia es importante porque podría contener la respuesta a la pregunta original.
- El unificador debe ser lo más general posible.

Ejemplo:

Unificar:	p (X, a, rojo(Y)) p (W, a, rojo(verde(W)))
Substituciones:	{X/W, Y/verde(W)}

Métodos de razonamiento: Unificación

- Encuentra el unificador más general. Si no existe, explique por qué.

  distancia (f (Y), Y, X)
  distancia (X, g (austin), f (Z))

- Sustituciones: {X / f (g (austin)), Y / g (austin), Z / g (austin)}

  distancia (f (g (austin)), g (austin), f (g (austin)))
  distancia (f (g (austin)), g (austin), f (g (austin)))

Métodos de razonamiento: Unificación

- Encuentra el unificador más general. Si no existe, explique por qué.

  X, Y y Z son variables; juan es una constante; h y n son funciones:

  1) ancestro (juan, X, h (X))

     antepasado (juan, Y, Y)

     •No hay un unificador general, porque cuando Y se sustituye por h (Y) o viceversa, creamos la expresión h (h (h (h ... ..)).

Métodos de razonamiento: Unificación

- Encuentra el unificador más general. Si no existe, explique por qué.
  X, Y y Z son variables; juan es una constante; h y n son funciones:

  2) ancestro (h (Y), Y, X)
  ancestro (X, n (juan), h (Z))

  {X/h(n(juan)), Y/n(juan), Z/n(juan)}

Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Semestre 2021 – II

Agenda

1. Métodos de razonamiento: Reglas de inferencia
2. Teorema de Resolución
3. Otros métodos de representación del conocimiento

Métodos de razonamiento: Reglas de inferencia

- Modus ponens
  - Dada una expresión de la forma p $\Rightarrow$ q, y otra expresión *p*, que son ambas verdaderas bajo una interpretación I, entonces modus ponens nos permite inferir que *q* también es verdadera bajo esa interpretación.
  - Modus ponens es una regla de inferencia sólida y correcta.
- Modus tolens
  - Si se sabe que p $\Rightarrow$ q es verdadero y también se sabe que *q* es falso, entonces podemos inferir que $\neg p$ es verdadero.

Métodos de razonamiento: Reglas de inferencia

- AND Elimination
  - Dada una expresión p $\wedge$ q, se sabe que es verdadera, entonces podemos inferir que *p, q* son ambas verdaderas.

- AND Introduction
  - Permite inferir la verdad de una conjunción basada en la verdad de sus elementos.
  - Si *p, q* son verdaderas, entonces podemos inferir que p $\wedge$ q también es verdadera.

Métodos de razonamiento: Reglas de inferencia

- Universal Instantiation
  - Si, cualquier variable universalmente cuantificada en una afirmación verdadera, se reemplaza por cualquier término del dominio, la afirmación resultante también es verdadera.

- Existential Instantiation
  - Si, cualquier variable cuantificada existencialmente en una afirmación verdadera, se reemplaza por un término apropiado del dominio, la afirmación resultante también es verdadera.

Métodos de razonamiento: Reglas de inferencia

1. $\forall X \; \forall Y$ caballo (X) $\wedge$ perro (Y) $\Rightarrow$ mas_rapido (X, Y)
2. $\exists Y$ greyhound (Y) $\wedge$ ($\forall Z$ conejo (Z) $\Rightarrow$ mas_rapido (Y, Z))
3. $\forall Y$ greyhound (Y) $\Rightarrow$ perro (Y)
4. $\forall X \; \forall Y \; \forall Z$ mas_rapido (X, Y) $\wedge$ mas_rapido (Y, Z) $\Rightarrow$ mas_rapido (X, Z)
5. caballo (harry)
6. conejo (ralph)

Objective

- mas_rapido (harry, ralph)

Métodos de razonamiento: Reglas de inferencia

7. greyhound (greg) $\wedge$ ($\forall$Z conejo (Z) $\Rightarrow$ mas_rapido (greg, Z))	2, EI
8. greyhound (greg)	7, AE
9. $\forall$Z conejo (Z) $\Rightarrow$ mas_rapido (greg, Z)	7, AE
10. conejo (ralph) $\Rightarrow$ mas_rapido (greg, ralph)	9, UI
11. mas_rapido (greg, ralph)	10, 6, MP
12. greyhound (greg) $\Rightarrow$ perro (greg)	3, UI
13. perro (greg)	12, 8, MP
14. caballo (harry) $\wedge$ perro (greg) $\Rightarrow$ mas_rapido (harry, greg)	1, UI
15. caballo (harry) $\wedge$ perro (greg)	5, 13, AI
16. mas_rapido (harry, greg)	14, 15, MP
17. mas_rapido (harry, greg) $\wedge$ mas_rapido (greg, ralph) $\Rightarrow$ mas_rapido (harry, ralph)	4, UI
18. mas_rapido (harry, greg) $\wedge$ mas_rapido (greg, ralph)	16, 11, AI
19. mas_rapido (harry, ralph)	17, 18, MP

Métodos de razonamiento: Teorema de Resolución

- Teorema de resolución
  - El principio de resolución fue introducido en 1965 por R. Robinson.
  - Es una técnica para probar teoremas usando cálculo de predicados.
  - Es un método de inferencia sólido, correcto y completo.
  - Proporciona una forma de encontrar contradicciones en una base de cláusulas con un conjunto mínimo de sustituciones.

Métodos de razonamiento: Teorema de Resolución

- El proceso de resolución implica los siguientes pasos:

1. Convertir las aserciones o axiomas en forma de cláusula.
2. Agregar la negación de lo que debe probarse (la hipótesis), en forma de cláusula, al conjunto de axiomas originales.
3. Resolver estas cláusulas, produciendo nuevas cláusulas que lógicamente provengan de las originales.
4. Producir o encontrar contradicciones, genere la cláusula vacía.
5. Extraer el resultado de las sustituciones.

Métodos de razonamiento: Teorema de Resolución

1. Conversión a forma de cláusula:

   - Todas las aserciones deben convertirse en su forma estándar - forma de cláusulas.
   - Formar conjunción de disyunciones.
   - Conjunción porque se supone que todas las cláusulas de la base de datos son verdaderas al mismo tiempo.
   - Disyunción porque cada cláusula individual está conectada con el operador “o” "$\vee$".

Métodos de razonamiento: Teorema de Resolución

1. Eliminar "$\Rightarrow$" usando la forma equivalente: $\neg$, $\vee$

2. Reducir la cobertura de la negación. Usando las transformaciones:

$\neg \exists X\ p(X) \equiv \forall X \neg p(X)$

$\neg \forall X\ p(X) \equiv \exists X \neg p(X)$

$\neg (P \vee Q) = (\neg P \wedge \neg Q)$

$\neg (P \wedge Q) = (\neg P \vee \neg Q)$

$P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$

$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R)$

$(P \wedge Q) = (Q \wedge P)$

$(P \vee Q) = (Q \vee P)$

$((P \wedge Q) \wedge R) = (P \wedge (Q \wedge R))$

$((P \vee Q) \vee R) = (P \vee (Q \vee R))$

Métodos de razonamiento: Teorema de Resolución

3. Estandarice las variables: cada cuantificador debe admitir solo un tipo de variable.

4. Mueva todos los cuantificadores a la izquierda, sin cambiar el orden.

5. Eliminar los cuantificadores existenciales, usar skolemization

6. Eliminar los cuantificadores universales

Métodos de razonamiento: Teorema de Resolución

7. Convierta la expresión en una conjunción de disyunciones

   Usar las propiedades de $\vee$ y $\wedge$

   $a \vee (b \vee c) \equiv (a \vee b) \vee c$

   $a \wedge (b \wedge c) \equiv (a \wedge b) \wedge c$

   $a \vee (b \wedge c) \equiv (a \vee b) \wedge (a \vee c)$

8. Llame a cada término en la conjunción como una cláusula separada

9. Estandarizar todas las variables.

Artificial Intelligence

ENRIQUE PELÁEZ

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Semestre 2021 – II

Agenda

1. Otros métodos de representación del conocimiento
2. Métodos de razonamiento artificial: Reglas de inferencia
3. Teorema de Resolución

Métodos de razonamiento: Unificación

- Para permitir la libertad en las sustituciones, todas las variables deben cuantificarse universalmente.
- Las variables cuantificadas existencialmente pueden eliminarse reemplazándolas con valores constantes que hacen que la expresión sea verdadera.

$\forall$ X $\exists$ Y mama (Y, X)

Y depende del valor de X

- Skolemization es una acción que reemplaza cada variable cuantificada existencialmente con una función que devuelve el valor constante apropiado del rango, en función de otra variable.

Métodos de razonamiento: Unificación

- En nuestro ejemplo, Y puede ser reemplazado por una función skolem *f* (X).

$$\forall\ X\ \text{mama}\ (f\ (X),\ X)$$

$$\forall X\ \forall Y\ \exists Z\ p\ (X, Y, Z) \rightarrow \forall X\ \forall Y\ p\ (X, Y, g(X, Y))$$

$$\forall Y\ \exists W\ \forall X\ \exists Z\ p\ (W, X, Y, Z) \rightarrow \forall Y\ \forall X\ p\ (h(Y), X, Y, f(Y, X))$$

$$\exists Y\ \forall X\ \forall W\ \forall Z\ p\ (Z, W, X, Y) \rightarrow \forall X\ \forall W\ \forall Z\ p\ (Z, W, X, abc)$$

Métodos de razonamiento: Reglas de inferencia

1. Modus ponens
2. Modus tolens
3. AND Elimination
4. AND Introduction
5. Universal Instantiation
6. Existential Instantiation

Métodos de razonamiento: Teorema de Resolución

- Teorema de resolución
  - El principio de resolución fue introducido en 1965 por John Alan Robinson.
  - Es una técnica para probar teoremas usando cálculo de predicados.
  - Es un método de inferencia sólido, correcto y completo.
  - Proporciona una forma de encontrar contradicciones en una base de cláusulas, con un conjunto mínimo de sustituciones.

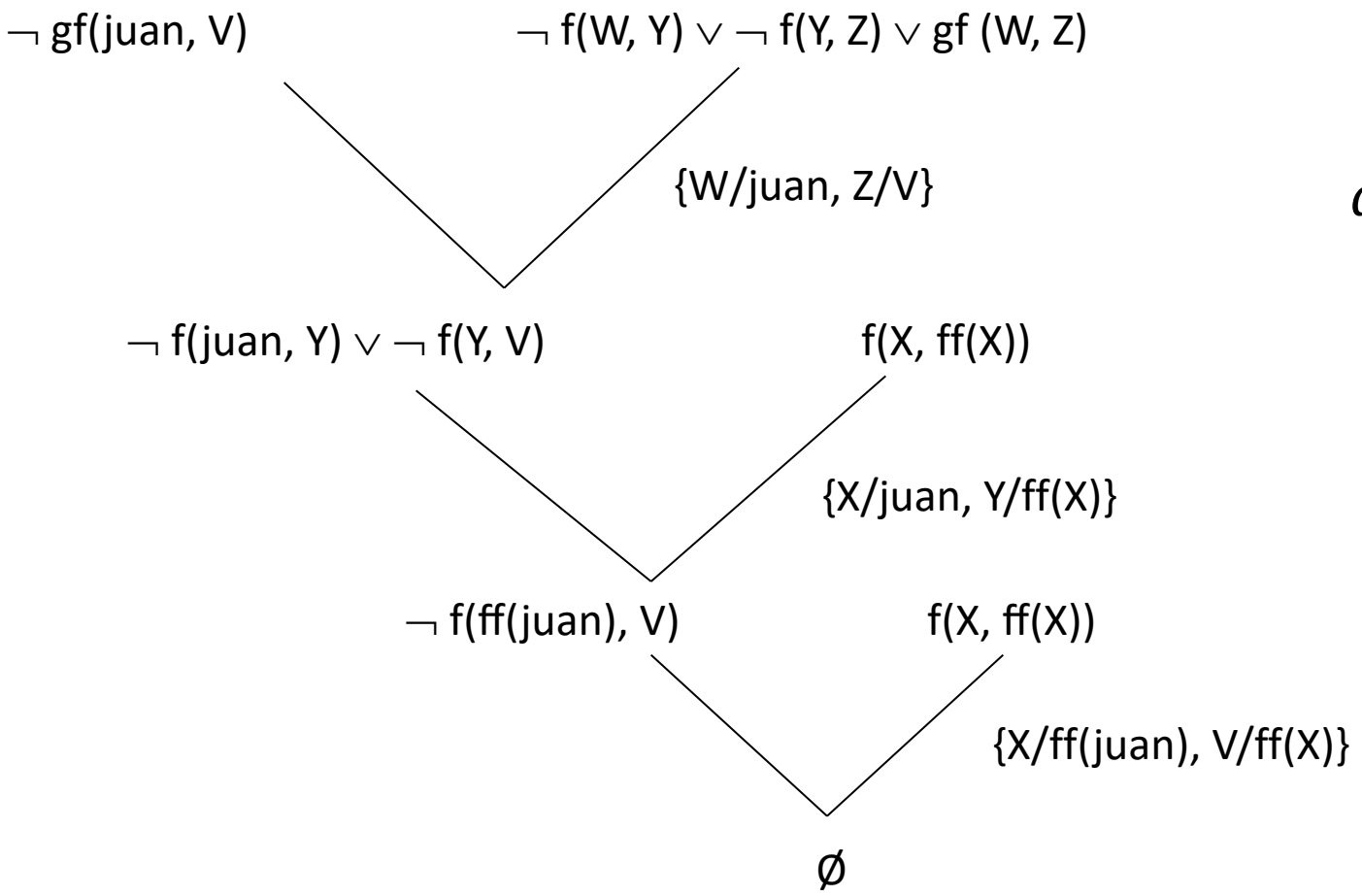
Métodos de razonamiento: Teorema de Resolución

- La aplicación de la regla de resolución implica los siguientes pasos:

1. Convertir las expresiones o axiomas en forma de cláusula.
2. Agregar la negación de lo que debe probarse (la hipótesis), en forma de cláusula, al conjunto de axiomas originales.
3. Resolver estas cláusulas, produciendo nuevas cláusulas que lógicamente provengan de las originales.
4. Producir o encontrar contradicciones; esto es, generar la cláusula vacía.
5. Extraer el resultado desde las sustituciones.

Métodos de razonamiento: Teorema de Resolución

1. f(X, ff(X))
2. ¬ f(W, Y) ∨ ¬ f(Y, Z) ∨ gf (W, Z)
3. ¬ gf(juan, V)



$$a \wedge \neg a \equiv \emptyset$$

Métodos de razonamiento: Teorema de Resolución

1. Conversión a forma de cláusula:

   - Todas las expresiones deben convertirse en su forma estándar - forma de cláusulas.
   - Formar conjunción de disyunciones.
   - Conjunción porque se supone que todas las cláusulas de la base de datos son verdaderas al mismo tiempo.
   - Disyunción porque cada cláusula individual está conectada con el operador “o” "$\vee$".

Métodos de razonamiento: Teorema de Resolución

1. Eliminar "$\Rightarrow$" usando la forma equivalente: $\neg$, $\vee$

2. Reducir la cobertura de la negación. Usando las transformaciones:

$\neg \exists X\ p(X) \equiv \forall X \neg p(X)$

$\neg \forall X\ p(X) \equiv \exists X \neg p(X)$

$\neg (P \vee Q) = (\neg P \wedge \neg Q)$

$\neg (P \wedge Q) = (\neg P \vee \neg Q)$

$P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$

$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R)$

$(P \wedge Q) = (Q \wedge P)$

$(P \vee Q) = (Q \vee P)$

$((P \wedge Q) \wedge R) = (P \wedge (Q \wedge R))$

$((P \vee Q) \vee R) = (P \vee (Q \vee R))$

Métodos de razonamiento: Teorema de Resolución

3. Estandarizar las variables: cada cuantificador sólo con un tipo de variable.

4. Mueva todos los cuantificadores a la izquierda, sin cambiar el orden.

5. Eliminar los cuantificadores existenciales, usar skolemization

6. Eliminar los cuantificadores universales

Métodos de razonamiento: Teorema de Resolución

7. Convertir la expresión en una conjunción de disyunciones

   Usar las propiedades de $\vee$ y $\wedge$

   $a \vee (b \vee c) \equiv (a \vee b) \vee c$

   $a \wedge (b \wedge c) \equiv (a \wedge b) \wedge c$

   $a \vee (b \wedge c) \equiv (a \vee b) \wedge (a \vee c)$

8. Llamar a cada término, en la conjunción, como una cláusula separada

9. Estandarizar todas las variables.

Métodos de razonamiento: Teorema de Resolución

Convertir en clausulas la siguiente expresión:

$\forall X ([a(X) \wedge b(X)] \Rightarrow [c(X,I) \wedge \exists Y(\exists Z [c(Y,Z)] \Rightarrow d(X,Y))]) \vee \forall X (e(X))$



paso 1: $\forall X(\neg[a(X) \wedge b(X)] \vee [c(X,I) \wedge \exists Y(\neg\exists Z[c(Y,Z)] \vee d(X,Y))]) \vee \forall X(e(X))$

paso 2: $\forall X([\neg a(X) \vee \neg b(X)] \vee [c(X,I) \wedge \exists Y(\forall Z[\neg c(Y,Z)] \vee d(X,Y))]) \vee \forall X(e(X))$

paso 3: $\forall X([\neg a(X) \vee \neg b(X)] \vee [c(X,I) \wedge \exists Y(\forall Z[\neg c(Y,Z)] \vee d(X,Y))]) \vee \forall W(e(W))$

paso 4: $\forall X\exists Y\forall Z\forall W([\neg a(X) \vee \neg b(X)] \vee [c(X,I) \wedge (\neg c(Y,Z) \vee d(X,Y))]) \vee (e(W))$

paso 5: $\forall X\forall Z\forall W([\neg a(X) \vee \neg b(X)] \vee [c(X,I) \wedge (\neg c(f(X),Z) \vee d(X,f(X)))]) \vee (e(W))$

paso 6: $([\neg a(X) \vee \neg b(X)] \vee [c(X,I) \wedge (\neg c(f(X),Z) \vee d(X,f(X)))]) \vee e(W)$

$a \vee (b \wedge c) \equiv (a \vee b) \wedge (a \vee c)$

paso 7: $[\neg a(X) \vee \neg b(X) \vee c(X,I)] \vee e(W) \wedge$

$[\neg a(X) \vee \neg b(X) \vee (\neg c(f(X),Z) \vee d(X,f(X)))] \vee e(W)$

paso 8: (i) $\neg a(X) \vee \neg b(X) \vee c(X,I) \vee e(W)$

(ii) $\neg a(X) \vee \neg b(X) \vee \neg c(f(X),Z) \vee d(X,f(X)) \vee e(W)$

paso 9: (i) $\neg a(X) \vee \neg b(X) \vee c(X,I) \vee e(W)$

(ii) $\neg a(U) \vee \neg b(U) \vee \neg c(f(U),Z) \vee d(U,f(U)) \vee e(V)$

Métodos de razonamiento: Teorema de Resolución

- Dos métodos para la extracción de resultados en resolución

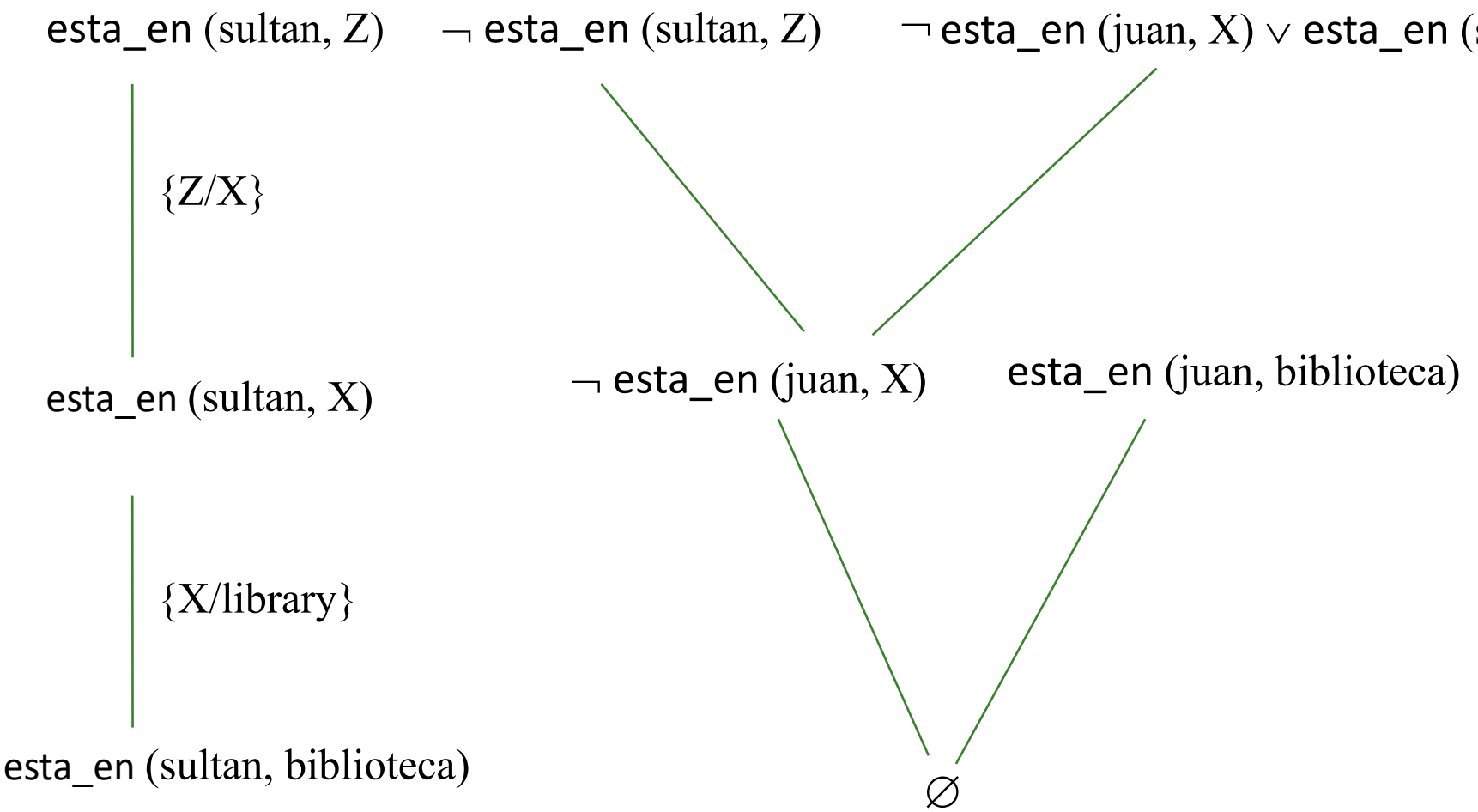
  1. Mantenga la conclusión o hipótesis original, y rastree en esta expresión, cada una de las sustituciones realizadas durante el proceso de resolución.

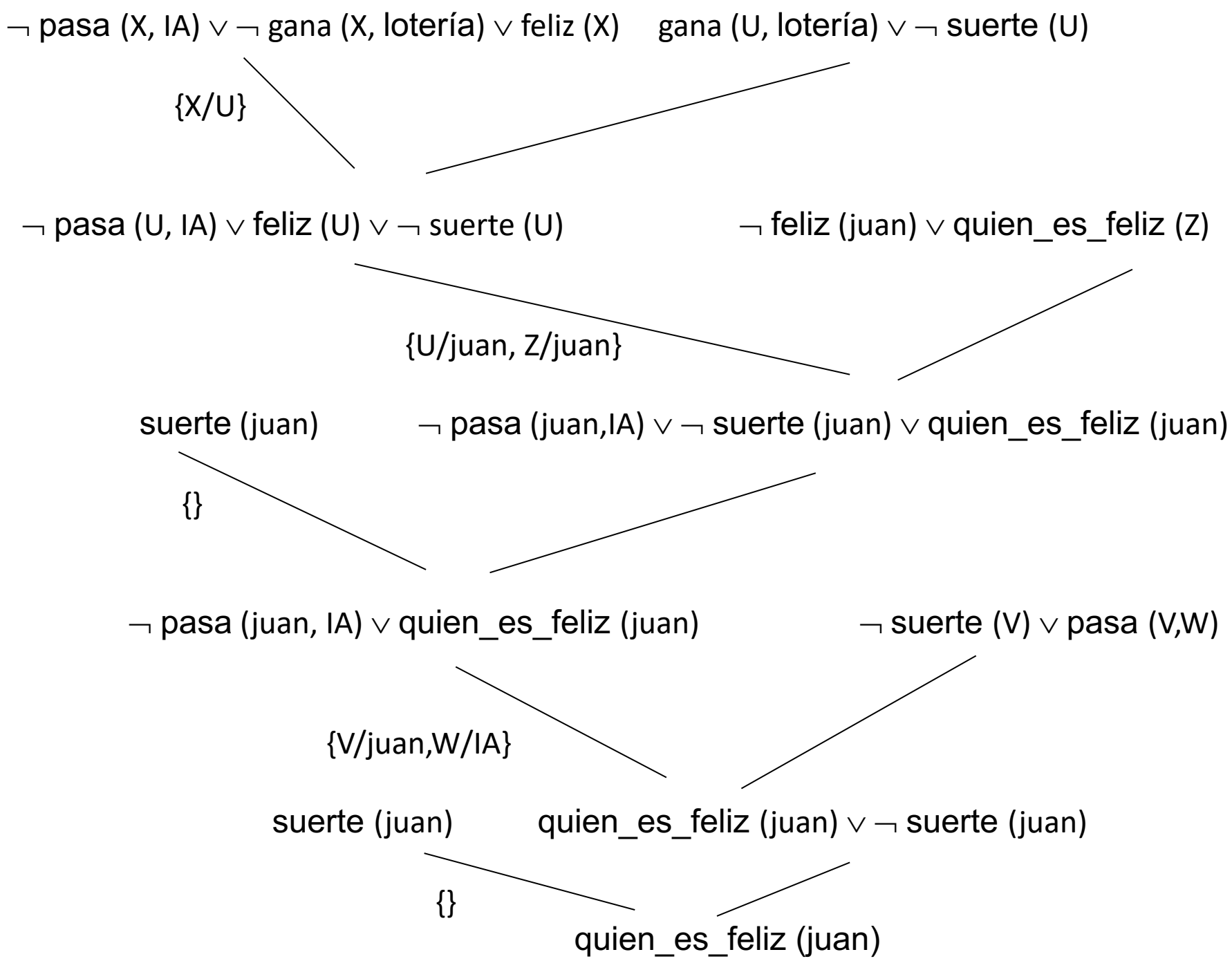
  2. Agregue a la negación de la hipótesis, un elemento con igual número de variables que la hipótesis, y realice un seguimiento de todas las sustituciones realizadas durante el proceso de resolución, en estas variables.

  Ejemplo:

  $$\neg \text{ objetivo (X, Y)} \vee \text{resultado (W, Z)}$$

Métodos de razonamiento: Teorema de Resolución




¬ pasa (X, IA) ∨ ¬ gana (X, lotería) ∨ feliz (X)
gana (U, lotería) ∨ ¬ suerte (U)
{X/U}
¬ pasa (U, IA) ∨ feliz (U) ∨ ¬ suerte (U)
¬ feliz (juan) ∨ quien_es_feliz (Z)
{U/juan, Z/juan}
suerte (juan)
¬ pasa (juan,IA) ∨ ¬ suerte (juan) ∨ quien_es_feliz (juan)
{}
¬ pasa (juan, IA) ∨ quien_es_feliz (juan)
¬ suerte (V) ∨ pasa (V,W)
{V/juan,W/IA}
suerte (juan)
quien_es_feliz (juan) ∨ ¬ suerte (juan)
{}
quien_es_feliz (juan)